arXiv:2610.01939v2 [cs.CV] 13 Oct 2026

Fewer Tokens, Better Action: GPT-6 Astra Robot Agents with 14% Higher Success Rate but 65% Fewer Tokens

Ruiyang Si[1] **Jianxin Bi**[1,2] **Shunyu Yang**[1] **Rui Ni**[1] **Wenbo Huang**[1]
Qiang Wang[1] **Shulong Jiang**[1] **Yintianrun Zhang**[1] **Feixiang Ruan**[1] **Kewei Zhang**[1]
Duomin Wang[3] **Xiuyu Li**[4] **Haiwen Feng**[4] **Zhen Dong**[3] **Daquan Zhou**[1]

[1]Peking University, [2]National University of Singapore, [3]NVIDIA, [4]Impossible Research

Abstract

Vision-language model (VLM) agents can control robots through visual feedback and action primitives, but repeated LLM calls and redundant observations incur substantial token overhead. We introduce PyRUA-Lean, an interactive code-execution framework in which the agent composes robot primitives, including vision-language-action (VLA) policies, into Python cells that check and retry locally and return only the images and state they request. Across 700 simulated task instances from LIBERO-PRO, RoboTwin 2.0, and RoboCasa365, it raises success from 63.1% to 71.7% over tool calling with the same GPT-6 Astra planner and primitives, using 49% fewer LLM calls and 65% fewer input tokens on jointly solved instances. On RoboDojo, without any VLA policy or privileged information, it reaches 51.6% average success rate (Score 58.2), 19.4 points above the top entry of the public leaderboard. Based on the PyRUA-Lean framework, we train PyRUA-Lean Coder v0.1-4B, an open 4B model, with GRPO on 45 LIBERO-PRO instances; it improves from 18.8% to 44.1% on unseen task instances, surpassing the Qwen3.5-27B base model.

Project Page: https://dagroup-pku.github.io/PyRUA-Lean/

GitHub Repo: https://github.com/DAGroup-PKU/PyRUA-Lean

Corresponding Author: Daquan Zhou

1 Introduction

Vision-language models (VLMs) can perform robot manipulation by interpreting visual observations and robot state, then invoking motion, perception, and grasping primitives, including vision-language-action (VLA) policies [1, 13, 38, 48]. Their efficiency depends partly on how these primitives are exposed to the planner. In a tool-calling interface, an operation that depends on a previous tool's result generally requires another LLM call. Intermediate results also accumulate in the conversation and are processed again in every later LLM call; in systems such as RPent [48], motion primitives automatically return multiple camera images at the end of each step of execution. Tasks requiring repeated localization, motion, and recovery can therefore incur substantial inference overhead, even when individual primitives are reliable.

Executable code can reduce this overhead. A program can compose primitives, inspect their outputs, and execute conditional branches or retries before returning control to the LLM. Code-based robot control is well established: Code as Policies and ProgPrompt generate robot programs from language instructions [17, 36],

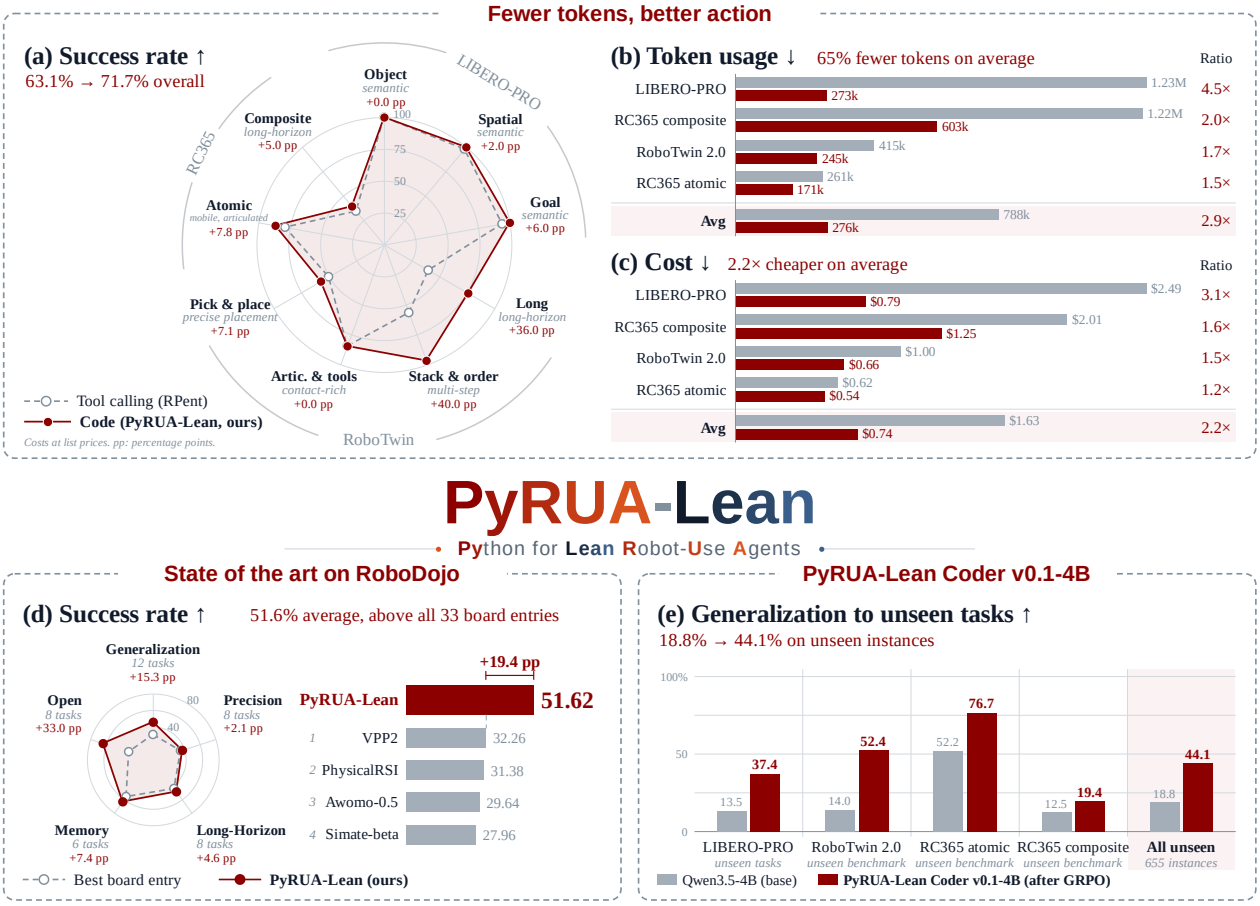


Figure 1 PyRUA-Lean at a glance. *Top, the interface against tool calling* with the same GPT-6 Astra planner and primitives: *(a)* success rate on all 700 instances, tagged by task property (semantic: perturbed layouts, objects, goals), RoboTwin 2.0 split by skill from task names; *(b, c)* input tokens and GPT-6 Astra list-price dollars per solved episode, on instances both agents solved (Avg: mean over all of them). *Bottom left, RoboDojo, (d):* success rate per capability dimension against the best entry of the public leaderboard in each dimension, and the average success rate of PyRUA-Lean and of the four highest-ranked of the board's 33 entries. *Bottom right, PyRUA-Lean Coder v0.1-4B, (e):* generalization. Qwen3.5-4B is trained with GRPO on 45 LIBERO-PRO task instances only; shown is its success rate on tasks and benchmarks it never saw, before (gray) and after training (red). All unseen pools these 655 instances. RC365: RoboCasa365.

while more recent systems incorporate execution feedback or refine programs across trials [10, 20]. Outside robotics, code-based agent interfaces have shown benefits in task completion and context efficiency [2, 39, 41]. These findings motivate our central question: *how can an LLM agent compose robot primitives to reliably complete tasks while reducing token consumption through selective observation and context management?*

We introduce **PyRUA-Lean** (Python for Lean Robot-Use Agents), an interactive code-execution interface for feedback-driven primitive composition and selective observation. The agent generates Python programs that compose robot primitives, check intermediate outcomes, and condition subsequent actions on execution feedback. Intermediate results remain available in a persistent program state, while only explicitly printed outputs and requested camera images are returned to the LLM. This allows the agent to handle intermediate execution steps without additional LLM calls and to control which observations enter its context.

This report studies the interface from three sides (Figure 1). *First*, we compare it with tool calling under controlled conditions. PyRUA-Lean builds on RPent's robot stacks and primitive implementations [48], and we compare it with RPent's tool-calling agent under the same GPT-6 Astra planner, frozen VLA policies, evaluation task instances, and LLM-call budgets, on 700 simulated task instances from LIBERO-PRO,

RoboTwin 2.0, and RoboCasa365. Both agents operate without cross-episode memory, and the comparison evaluates the interfaces as a whole, including their support for control flow, persistent program state, and observation delivery. *Second*, we ask how far the interface carries a frozen LLM on a benchmark built for pretrained robot policies. On RoboDojo [8], we give the same planner a small library of geometric primitives that we wrote ourselves, and follow the benchmark's evaluation protocol. The agent uses no VLA policy, no demonstrations, no fine-tuning, and no privileged information: everything it observes is the RGB observation that the benchmark sends to every policy. *Third*, we ask whether a small open model can learn the interface. Based on the PyRUA-Lean framework, we train Qwen3.5-4B [30] to write the cells itself, using GRPO [35] with task success as the only reward, and call the resulting planner PyRUA-Lean Coder v0.1-4B.

Our contributions can be summarized as:

- **PyRUA-Lean, a token-efficient code interface for robot agents.** The agent composes robot primitives, VLA policies included, into Python cells that check outcomes, keep state, and return only the observations they request. With the same planner and primitives as tool calling, it raises success from 63.1% to 71.7% and, on jointly solved instances, uses 49% fewer LLM calls and 65% fewer input tokens (Section 3).
- **State-of-the-art results on RoboDojo.** With GPT-6 Astra as the planner, no learned policy acting on the robot, and no privileged information, PyRUA-Lean reaches 51.6% average success rate (Score 58.2), 19.4 points above the top entry of the public leaderboard and 2.3 times the same LLM controlling the end-effector directly, and is the highest on all five capability dimensions (Section 4).
- **PyRUA-Lean Coder v0.1-4B, an open planner trained to use the interface.** Using the PyRUA-Lean framework, we train Qwen3.5-4B with GRPO on only 45 LIBERO-PRO instances. On tasks and benchmarks it never saw, its success rate rises from 18.8% to 44.1%, above the Qwen3.5-27B base model; on RoboTwin 2.0 alone, from 14.0% to 52.4%. We will release the weights (Section 5).

2 Related Work

LLM-based robot control. Language models have been used to select robot skills, interpret execution feedback, and generate motion commands. SayCan grounds skill selection in language instructions and learned affordances [1], while Inner Monologue incorporates environment feedback into planning [13]. More recent systems support finer-grained control: Show-Harness exposes discrete semantic actions to a VLM [9], and GPT-6 Astra has been evaluated as an embodied policy that generates or corrects robot actions [38]. VLA models provide learned visuomotor capabilities for language-conditioned control [4, 5, 16, 29]. Harness VLA integrates frozen VLA policies with analytic primitives through RPent, a memory-guided agent framework [48]. PyRUA-Lean builds on RPent's robot stacks and primitive implementations, exposing these capabilities through an interactive Python interface. We compare this interface with RPent's tool-calling interface using the same planner and underlying primitives, with cross-episode memory disabled in both agents.

Code-based robot control. Code as Policies and ProgPrompt generate executable robot programs from language instructions [17, 36], while VoxPoser generates code that constructs spatial value maps for motion planning [14]. Subsequent work explores execution feedback and program adaptation. CaP-X benchmarks coding agents across primitive abstraction levels and single- or multi-turn interaction modes [10], and VLCP periodically regenerates a control function from updated observations within an episode [23]. Other systems accumulate reusable capabilities across trials: Voyager develops a code-based skill library in Minecraft [40], ASPIRE refines robot programs and consolidates experience into reusable skills [20], and RoboRSI converts established tool-use workflows into executable routines as part of a robot self-improvement system [25]. These studies establish program synthesis, feedback-driven execution, and skill reuse as complementary approaches to embodied control. Our focus is the success and inference cost of interactive code execution relative to tool calling over the same robot primitives, including VLA policies. We evaluate this comparison without cross-episode skill accumulation, while allowing state and code reuse within each episode.

Code-based interfaces for efficient agents. Research on general-purpose agents provides a broader motivation for this comparison. ReAct interleaves reasoning with environment interaction [45], and Toolformer studies the use of external APIs by language models [34]. CodeAct shows that executable Python can improve agent performance relative to text or JSON action interfaces [41], while SWE-agent demonstrates the importance of

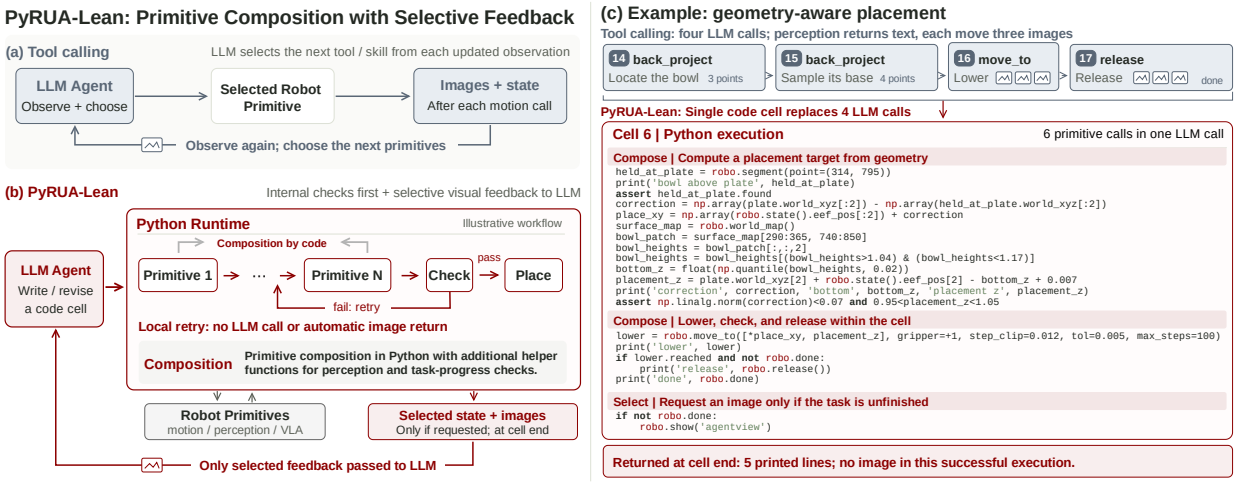


Figure 2 PyRUA-Lean combines primitive composition with selective feedback. (a) The tool-calling agent invokes robot primitives through tool calls. Operations that depend on preceding results generally require another LLM call, and motion calls automatically return images and state. (b) PyRUA-Lean instead generates Python cells that compose robot primitives with helper functions, conditional checks, and local retries. Intermediate execution stays within the runtime, while explicitly requested images and state messages are recorded and returned at cell end for replanning. (c) A recorded placement example illustrates both mechanisms: one PyRUA-Lean cell replaces the baseline's LLM calls 14–17, reducing four LLM calls to one. The cell computes a placement target from scene geometry (compose), conditionally executes lowering and release (compose), and requests an image only if the task remains unfinished (select). This successful execution returns five printed lines and no image, demonstrating how programmatic composition and selective feedback reduce repeated LLM calls. Appendix A follows the whole episode.

interface design for software-engineering agents [44]. More recent systems use code to compose tool calls and process intermediate results outside the model context, reducing the information returned to the LLM [2, 39]. PyRUA-Lean applies these interface principles to robot manipulation, where observations are visual and primitive execution may fail. We measure task success together with LLM-call usage, input-token consumption, and inference cost, and analyze how programmatic composition and observation passing contribute to the gains.

3 PyRUA-Lean: Primitive Composition with Selective Observation

PyRUA-Lean is an interactive code-execution interface for LLM-based robot agents. It supports two complementary mechanisms: composing robot primitives into programs that respond to execution feedback, and controlling which intermediate results and visual observations enter the LLM context. The interface builds on RPent's robot stacks and primitive implementations [48], including classical motion and perception primitives and learned VLA policies.

3.1 Robot Interface and Execution Model

The robot is exposed as a Python object, `robo`, whose methods invoke primitives and return structured results, such as object positions and motion outcomes. The agent receives an API reference (Table 1 lists the primitives) and interacts with the robot through a `python(code)` tool. In an LLM call, the model generates a code cell; the runtime executes it, and the returned feedback informs the next call. GPT-6 Astra writes one cell per LLM call, but a call may hold several (Section 5.3). A persistent Python namespace retains variables and helper functions across cells within an episode. Our evaluation uses no cross-episode memory.

An uncaught exception terminates the current cell and returns its traceback to the LLM. Variables assigned before the exception remain in the persistent namespace, allowing the agent to revise the failed operation in a subsequent cell. Before each primitive that acts on the robot, the runtime terminates the cell if the task has already succeeded. An external watchdog enforces the two-hour episode limit by interrupting any active primitive and stopping the planner. We impose no separate per-cell execution limit; in our setup, the Codex

Table 1 The two agents' interfaces: each primitive is an RPent tool for tool calling and a `robo` method with the same name and arguments for code, both running RPent's implementation. Names pooled over the benchmarks.

	Tool (RPent)	*Code* (PyRUA-Lean)
Primitives, the same in both		
Arm motion	`move_to`, `move_pose`, `move_delta`, `rotate_wrist`, `rotate_pitch`	
Gripper	`set_gripper`, `release`, `scripted_grasp`	
VLA policy	`pi0_pick`, `pi0_doubled`, `lingbot_act`, `rldx_skill`, `rldx_arm`	
Mobile base	`navigate_to`, `move_base`	
Perception	`segment`, `back_project`, `back_project_batch`, `sample_world_xyz`, `query_world_map`	
Observations		
State, success flag	with every move, or `view_env_state`	`robo.state()`, `robo.done`
Camera images	three with every move	when the code calls `robo.show`
World coordinates	through the perception tools	the same, or `robo.world_map()` as one array

CLI returns a tool-call timeout after 300 s without cancelling the cell, and subsequent cells are queued.

3.2 Feedback-Driven Primitive Composition

Each cell can compose multiple primitives and condition subsequent operations on their outcomes. For example, a program may approach an object, attempt a grasp only if the approach succeeds, and retry or stop based on the resulting state. These checks and branches execute within the cell without additional LLM calls. At the end of the cell, the returned feedback allows the LLM to revise subsequent actions, combining program-level feedback with LLM-level replanning.

Beyond composing primitive calls, programs can *compose* task-specific computations and control routines from existing observations and primitives, as illustrated by the geometric computation in Figure 2(c). For example, the agent computes the bowl's base height from a world-coordinate map to determine a placement target that no individual primitive directly returns. It can also define helper functions for repeated operations, such as a guarded descent. Computed values and helper functions remain available to later cells through the persistent namespace.

3.3 Selective Observation and Context Management

Primitive results remain available in the runtime without being individually added to the LLM conversation. The agent specifies which feedback to return through explicit output statements and camera requests such as `robo.show`. Requested images and state messages are recorded at the points specified in the code and returned together at the end of the cell. In the episode summarized in Appendix A, cell 5 requests a camera image after transporting the bowl. The placement cell shown in Figure 2(c) requests an image only if the task remains unfinished and therefore returns no images after successful placement.

Programmatic composition reduces the LLM calls needed for dependent operations, while selective feedback limits the outputs and images added to subsequent prompts. Persistent program state retains intermediate data outside the conversation. These mechanisms control new context rather than removing existing history: both agents run in the Codex CLI [26], which keeps the full conversation and compacts it, summarizing its earlier part, only when it approaches 272k tokens.

3.4 Evaluation Against Tool Calling

Our central question is whether an LLM agent can compose robot primitives to improve task completion while reducing token overhead through programmatic execution and selective feedback. We hold the planner, the primitives, and the task instances fixed, and evaluate PyRUA-Lean against a tool-calling baseline through three hypotheses (**H**):

- **H1:** PyRUA-Lean achieves higher task success than tool calling under equal LLM-call budgets.
- **H2:** PyRUA-Lean requires fewer LLM calls and input tokens to solve tasks.
- **H3:** PyRUA-Lean retains its token efficiency when VLA policies or operating guides are absent.

Table 2 Main results. Success rate on all task instances; LLM calls, input tokens and dollars at list prices per solved episode, on the task instances both agents solved (means). *Tool*: tool calling (RPent); *Code*: PyRUA-Lean (ours). Shaded: code's gain, in points (Δ) or how many times fewer (×). *All* pools the four benchmark groups, where LIBERO-PRO's long episodes dominate the token factor. RC365: RoboCasa365.

	All task instances			Per solved episode, on the task instances both agents solved								
	Success rate (%)			LLM calls			Tokens			Cost ($)		
Benchmark	*Tool*	*Code*	Δ	*Tool*	*Code*	×	*Tool*	*Code*	×	*Tool*	*Code*	×
LIBERO-PRO	83.0	94.0	+11.0	21.4	8.4	2.5×	1.23M	273k	4.5×	2.49	0.79	3.1×
RoboTwin 2.0	60.0	69.2	+9.2	12.2	8.9	1.4×	415k	245k	1.7×	1.00	0.66	1.5×
RC365 atomic	78.9	86.7	+7.8	10.4	5.6	1.8×	261k	171k	1.5×	0.62	0.54	1.2×
RC365 composite	34.4	39.4	+5.0	28.4	14.9	1.9×	1.22M	603k	2.0×	2.01	1.25	1.6×
All	63.1	71.7	**+8.6**	17.0	8.7	**2.0**×	788k	276k	**2.9**×	1.63	0.74	**2.2**×

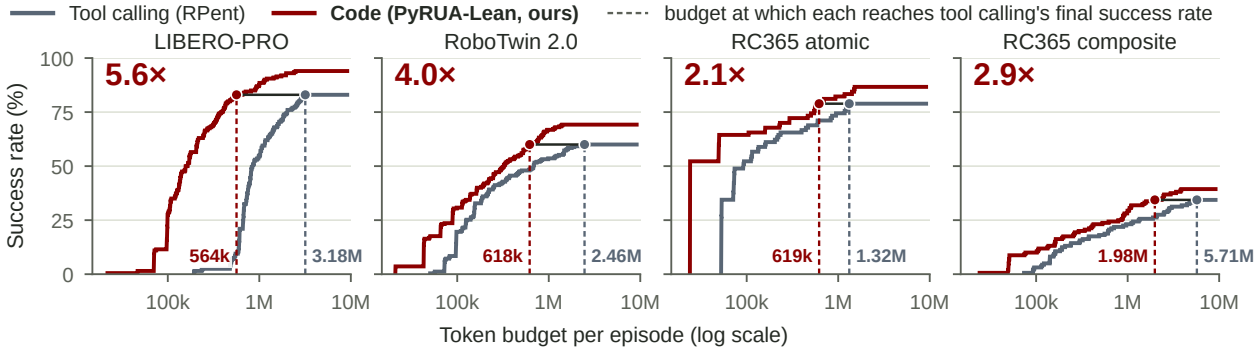


Figure 3 Success rate if every episode were stopped once its token consumption reached the budget on the horizontal axis. Dashed lines mark the budget at which each agent first reaches tool calling's final success rate, with its value on the axis; the large number is their ratio. RC365: RoboCasa365.

Benchmarks. We evaluate 40 tasks from LIBERO-PRO's four perturbed suites [19, 50], 50 dual-arm tasks from RoboTwin 2.0 [7], and 50 tasks from RoboCasa365's Target50 set [24]. Each task is evaluated at five environment seeds, with one episode per agent for each task–seed pair, yielding 700 paired task instances and 1,400 episodes. More implementation details can be found in Appendix F.

Agents. Both agents use GPT-6 Astra [27] at high reasoning effort through the Codex CLI, with the same robot stacks, primitive implementations, and frozen VLA policies: $\pi_{0.5}$ on LIBERO-PRO [29], LingBot-VLA on RoboTwin 2.0 [42], and RLDX-1 on RoboCasa365 [15]. Both use SAM 3 for segmentation [6]. The baseline invokes RPent's primitives through tool calls; PyRUA-Lean composes them through its Python interface. Both receive the same operating guides where available; RoboCasa365 has none. Both agents operate without cross-episode memory.

Metrics. Success rate is measured over all task instances using each benchmark's success check, with budgets of 40 LLM calls per episode, or 100 for RoboCasa365 composite tasks, and two hours. These budgets limit LLM calls, not the number of primitive executions. We compare LLM calls, cumulative input tokens, and estimated monetary cost on instances solved by both agents, reporting whole-episode means and ratios of aggregate totals. Counts include post-success LLM calls and cached input tokens. Cost-accounting details and all-episode costs are reported in Appendix E.

3.5 Task Success and Inference Cost

Task success improves across all benchmark groups. Table 2 and Figure 1(a–c) show that PyRUA-Lean increases overall success from 63.1% to 71.7%, an improvement of 8.6 percentage points, or approximately 14% relative. The gains are 11.0 points on LIBERO-PRO, 9.2 on RoboTwin 2.0, 7.8 on RoboCasa365 atomic tasks, and 5.0 on composite tasks. PyRUA-Lean alone solves 96 instances, whereas tool calling alone solves 36. These results support Hypothesis 1 within the evaluated setting: PyRUA-Lean completes more tasks under the same LLM-call budget. Figure 4 illustrates a recovery example in which PyRUA-Lean uses geometric

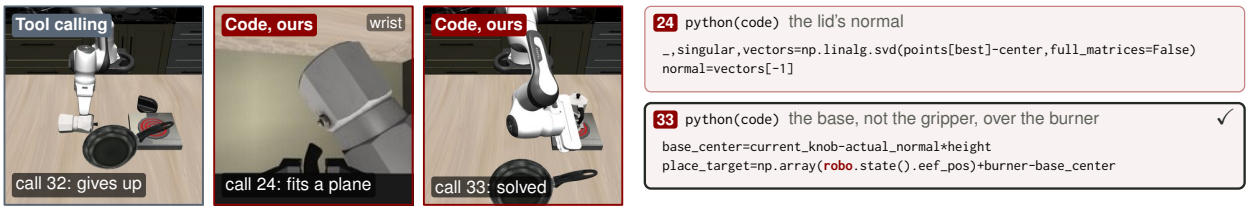


Figure 4 Only code solved this LIBERO-PRO instance ("turn on the stove and put the moka pot on it"): both agents dropped the pot, and the code agent fitted a plane to the lid, turned the pot upright and set its base over the burner (Appendix D).

computation to reposition a dropped object and complete the task. Appendix D provides further analysis of instances solved by only one agent.

Completing the same tasks requires fewer LLM calls and tokens. On jointly solved instances, PyRUA-Lean reduces mean LLM calls from 17.0 to 8.7 and input tokens from 788k to 276k, corresponding to reductions of 49% and 65%. Estimated cost decreases from \$1.63 to \$0.74 per episode. The smaller monetary reduction reflects, in part, caching discounts on the baseline's repeatedly processed history. These findings support Hypothesis 2 and show that the higher overall success rate is accompanied by lower inference overhead on shared successes. The pooled token reduction is influenced strongly by LIBERO-PRO's longer episodes; per-benchmark results are therefore also reported.

Figure 3 complements this comparison with retrospective token-budget cutoffs on recorded trajectories. On LIBERO-PRO, PyRUA-Lean reaches the baseline's final success rate with 564k tokens per episode, compared with 3.18M for tool calling. This connects success and efficiency beyond the jointly solved subset, although the agents were not rerun with these token budgets specified beforehand.

Section 4 evaluates the interface on a fourth benchmark, RoboDojo.

3.6 Analysis of LLM Calls and Observation Feedback

Fewer LLM calls account for most of the token reduction. Figure 5 decomposes the token ratio into the LLM-call ratio and the ratio of average input tokens per LLM call. On LIBERO-PRO, the 4.48× token ratio comprises 2.55× fewer LLM calls and 1.76× fewer tokens per LLM call. On RoboCasa365 atomic tasks, PyRUA-Lean's LLM calls are larger on average, yet fewer of them still reduce total token usage. Thus, Hypothesis 2 is supported primarily by reducing the number of LLM calls, rather than uniformly making each prompt smaller.

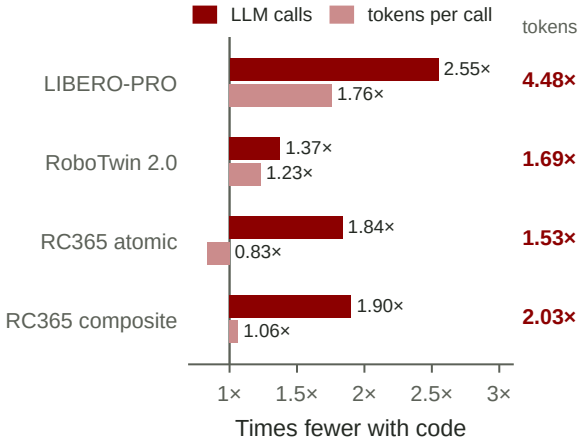


Figure 5 Token-savings decomposition on jointly solved instances. Tool-calling-to-PyRUA-Lean ratios quantify reductions in LLM calls (solid) and tokens per LLM call (light); their product gives the total token-reduction factor (right). Ratios below 1× indicate increased usage by PyRUA-Lean.

Execution traces illustrate how programmatic composition enables this reduction. A cell can execute dependent motions, check their outcomes, and retry a primitive before returning control to the LLM. In the bowl-placement example, the agent also computes placement geometry directly from observations. These behaviors address the central research question by moving intermediate coordination and computation into the execution runtime. The token decomposition is an accounting analysis, however, and does not independently isolate the causal contribution of each interface feature.

On-demand images alone do not reproduce the gains. On LIBERO-PRO, we additionally modify the

Table 3 Ablations: the VLA policy, the operating guides or both removed from *both* agents, on every task instance. Columns as in Table 2: success rate on all task instances, the other columns on the instances both agents solved in that setting; each group's first row is that table's. RoboCasa365 has no operating guides (–).

VLA	Guides	Success rate (%)			LLM calls			Tokens			Cost ($)		
		Tool	*Code*	Δ	*Tool*	*Code*	×	*Tool*	*Code*	×	*Tool*	*Code*	×
LIBERO-PRO													
✓	✓	83.0	94.0	+11.0	21.4	8.4	2.5×	1.23M	273k	4.5×	2.49	0.79	3.1×
✓	✗	84.0	88.0	+4.0	19.0	9.3	2.0×	901k	286k	3.1×	1.89	0.71	2.6×
✗	✓	73.0	80.5	+7.5	22.9	10.9	2.1×	1.34M	355k	3.8×	2.60	0.86	3.0×
✗	✗	74.0	82.0	+8.0	20.0	9.8	2.0×	943k	290k	3.2×	2.00	0.75	2.7×
RoboTwin 2.0													
✓	✓	60.0	69.2	+9.2	12.2	8.9	1.4×	415k	245k	1.7×	1.00	0.66	1.5×
✓	✗	64.0	66.8	+2.8	11.6	9.1	1.3×	392k	251k	1.6×	0.91	0.64	1.4×
✗	✓	42.8	68.8	+26.0	23.1	11.5	2.0×	1.03M	316k	3.2×	1.94	0.85	2.3×
✗	✗	60.4	66.4	+6.0	16.9	12.0	1.4×	620k	329k	1.9×	1.29	0.88	1.5×
RoboCasa365 atomic													
✓	–	78.9	86.7	+7.8	10.4	5.6	1.8×	261k	171k	1.5×	0.62	0.54	1.2×
✗	–	45.6	65.6	+20.0	26.5	18.7	1.4×	760k	627k	1.2×	1.48	1.37	1.1×
RoboCasa365 composite													
✓	–	34.4	39.4	+5.0	28.4	14.9	1.9×	1.22M	603k	2.0×	2.01	1.25	1.6×
✗	–	7.5	20.0	+12.5	58.5	31.1	1.9×	2.58M	1.14M	2.3×	5.50	3.08	1.8×

tool-calling baseline to return images only on request. As shown in Table 14, success decreases from 83.0% to 68.5%. On the 131 instances solved by all three configurations, the modified baseline makes more LLM calls and consumes approximately the same total input tokens as the original baseline. This suggests that reducing automatic visual feedback alone is insufficient: observation requests must be considered together with action execution and replanning. The experiment does not separately quantify the contribution of selective feedback within PyRUA-Lean.

3.7 Ablation Studies

Token savings persist without VLA policies or guides. We remove VLA policies, operating guides, or both from each agent on the same task instances (Appendix G); RoboCasa365 has no guides, so only VLA policies are ablated. Across these settings, PyRUA-Lean maintains higher overall success and lower input-token usage on jointly solved instances (Table 3), supporting Hypothesis 3. Without VLA policies, it achieves 68.8% success on RoboTwin 2.0 versus 42.8% for tool calling, with traces showing classical primitives composed into approach, gripper control, and state checks.

Guidance has interface-dependent effects. On RoboTwin 2.0 without VLA policies, removing guides increases tool-calling success from 42.8% to 60.4%, suggesting that guidance is not uniformly beneficial. Jointly solved subsets also vary across settings, so their token costs do not isolate component effects on a fixed task subset.

3.8 Failure Cases and Limitations

Improved budget use explains some, but not all, additional successes. Of the 96 instances solved only by PyRUA-Lean, the baseline exhausts its LLM-call budget in 52 and ends unsuccessfully before exhausting it in 44. The former cases are consistent with programmatic composition enabling more execution within the budget. The latter include examples of geometric reasoning and scripted recovery, but also cases where a VLA succeeds in one run and fails in the other. The success difference therefore cannot be attributed entirely to better recovery logic.

Efficiency gains depend on task structure and failures. Gains are smallest on RoboCasa365 atomic tasks, where a VLA policy often completes most of the task and the initial prompt accounts for much of the input. PyRUA-Lean's larger API description can offset part of the benefit from fewer LLM calls. Including failed episodes, tool calling uses 1.1× the input tokens of PyRUA-Lean on RoboCasa365 atomic tasks, and the estimated inference cost of the two agents is about the same (Appendix E). Efficiency on jointly solved

instances should therefore be distinguished from the cost of all attempts.

4 PyRUA-Lean on RoboDojo

Section 3 holds the planner and the primitives fixed and varies the interface. This section asks a different question: how far does the interface carry a frozen LLM on a public benchmark built for pretrained robot policies? RoboDojo [8] is such a benchmark, with a leaderboard led by VLA and world-action models [33]. Zhang et al. [47] recently showed that GPT-6 Astra, issuing one end-effector target per LLM call from RGB images, reaches 22.5% average success on it without any robot training, above every pretrained policy listed at the time but with almost no success where precision is needed. We evaluate the same LLM, on the same RGB observation, acting through PyRUA-Lean.

4.1 Benchmark and Protocol

RoboDojo's simulation track has 42 tabletop tasks for a dual-arm ARX-X5 station, grouped into five capability dimensions: Generalization (12 tasks), Precision (8), Long-Horizon (8), Memory (6), and Open (8). A task is evaluated on 50 public layouts per evaluation seed, and each seed has its own set of layouts; a Generalization task runs 25 episodes in its standard scene and 25 with randomized room and table appearance. One evaluation seed is therefore 2,100 episodes. The benchmark reports the success rate (SR), the percentage of episodes that meet the task's full-success criterion, and the Score, the mean process reward times 100, which gives partial credit for completed stages. As on the leaderboard, a task's values are averaged within its dimension, and the Average is the unweighted mean of the five dimension values.

We follow this protocol with RoboDojo's evaluation code (`scripts/robodojo.sh eval`, commit `cb2a37d`, with the XPolicyLab release it pins [43]). Our agent is packaged as an XPolicyLab policy, like any policy the benchmark evaluates: RoboDojo's evaluation client runs the simulation, sends observations, executes the returned actions, and judges success, and it ends an episode as soon as the success check passes or the task's step limit is reached. An observation is three RGB images, from a head camera and from a camera on each wrist, and the joint and end-effector state of the two arms. We compare with the entries of the public leaderboard as of October 9, 2026. We evaluate on one evaluation seed (seed 0), 2,100 episodes; the configuration with depth of Section 4.4 was evaluated on three.

4.2 System

One fixed agent runs all tasks. The planner is GPT-6 Astra at high reasoning effort, the model of Section 3 and of [47]; its weights are untouched and it sees no demonstrations. It acts only through `python(code)`, here in a minimal agent loop of our own around an OpenAI-compatible endpoint instead of the Codex CLI, with a budget of 100 LLM calls per episode (counted on the calls that run a code cell), images on demand, and no memory across episodes (settings in Appendix H).

Primitives. Where Section 3 runs RPent's primitive implementations, the primitives of the RoboDojo agent are our own: we wrote the library for the benchmark's dual-arm station, and Table 4 lists everything in it that the code can call. There is no pick, place, or grasp primitive: a grasp is a few lines of code that locate an object, move above it with the measured heading, descend along a line, and close the gripper while checking the reported width. Without a depth image, `locate` measures an object from its masks in the cameras that see it: the fixed head camera bounds its footprint on the table, and a wrist camera that also sees it settles its height, so a program often moves a wrist camera above an object and measures again before it grasps. We added a primitive only when a program could not build its function from the others.

No VLA policy. Unlike the agents of Section 3, this one has no VLA policy among its primitives, and that is a choice. A VLA policy is a stochastic primitive: the same call may succeed in one episode and fail in the next, and in Section 3 some of the instances that only one agent solved differ only in whether the same VLA call succeeded (Appendix D). We did not want this variance in the result. (The configuration with depth varies by a fraction of a point between evaluation seeds, Section 4.4.) The primitives of Table 4 are geometric and report what they measured or reached, so a program can check every step, and what the result measures is

Table 4 Our primitive library for RoboDojo: everything the agent's code can call. There is no VLA policy and no pick, place, or grasp primitive; SAM 3 is the only learned component besides the LLM. Every method works from the RGB images and the robot state that RoboDojo's evaluation client sends; none reads a depth image or the simulator's state.

	Method of `robo`	What it does
Motion	`move_to(xyz, arm, gripper, pitch, yaw, roll, path)`	Drives one arm's grasp point to a pose. The library solves the inverse kinematics; the path is a straight line of the grasp point or a joint-space swing. Reports whether the pose was reached, or which nearby poses are reachable.
	`set_gripper(arm, command, opening)`	Runs a full gripper stroke; reports whether the fingers stopped on something and how wide it is.
	`home(arm)`	Returns an arm to its initial joint angles.
Perception	`locate(text, camera)`	Segments the object named in text with SAM 3 in one camera's image, finds it in the other cameras, and measures it in 3D from these views: center, top, footprint, and the heading for a grasp across its narrow side. Reports how many cameras agreed: one camera cannot measure a height.
	`segment(text, camera)`, `back_project(row, col, camera)`	The mask of a named object; the world point at a pixel, on the object under it or on the table.
Observation	`state()`, `show(camera)`, `image(camera)`, `artifact(name)`, `camera_meta(camera)`	End-effector poses, gripper openings and joint angles; attach a camera image to the cell's feedback; the image as an array or as file bytes; a camera's calibration (a wrist camera's follows from its arm's state).

the programs the LLM writes. As in the LLM-as-policy setting of [47], no learned policy acts on the robot; unlike that setting, our agent has a perception primitive built on a segmentation model [6].

No privileged information. During an episode, everything the agent observes is the observation that RoboDojo's evaluation client sends to every policy: the three RGB images and the joint and end-effector state. It reads no depth image and nothing of the simulator's scene state: it is given no object poses and no description of the layout, so it measures where things are from the images, with `locate`. What the library knows in advance is the station itself, that is, the calibration of its cameras, where its arms stand, and the height of its table, which are the same in every episode. It also receives no success or progress signal while an episode runs: unlike the agents of Section 3, which can read the benchmark's success flag, it has to judge from the images and the robot state whether the task is done.

Prompt. The prompt holds the API reference, operating notes on how the tools behave (frames, reach, how the gripper reports a grasp, how to measure an object with the wrist cameras) and on how strictly the benchmark judges a final scene, and a task card with the task's name, the instruction the benchmark generates for the layout, and the first camera images. Beyond that instruction, it contains no description of a task and no recipe for solving one. The direct-control agent of [47], in contrast, receives for each task a description and its scoring stages, derived from the benchmark's wiki.

4.3 Results

PyRUA-Lean is above every entry of the board. Over 2,100 episodes (one evaluation seed), the agent reaches an Average SR of 51.6% and a Score of 58.2 (Table 5); pooled over all episodes, it solves 1,061 of 2,100 (50.5%). The highest-ranked entry of the board, VPP2, has 32.3% and 39.3; PhysicalRSI, the strongest agent entry, has 31.4% and 36.3. PyRUA-Lean has the highest success rate on all five dimensions (Table 5), and on 22 of the 42 tasks it is above every entry of the board (Appendix K).

Same LLM, another interface. Against GPT-6 Astra controlling the end-effector directly, the Average SR rises from 22.5% to 51.6%, by a factor of 2.3 (Figure 6). The largest differences are where direct control is weakest: Precision goes from 4.0% to 37.3% and Long-Horizon from 8.3% to 48.0%. PyRUA-Lean is higher on 34 of the 42 tasks and lower on 2. The two systems share the LLM and the RGB observation and differ in several things at once: the action interface (programs over primitives against one end-effector target per LLM call), perception (a segmentation-based `locate` against the images as the LLM reads them), reasoning effort (high against medium), and the prompt (tool notes against per-task descriptions). The gap therefore measures the

Table 5 RoboDojo-Sim in the leaderboard's format; each value is Score / SR (%). *First row:* PyRUA-Lean (ours) over one evaluation seed (2,100 episodes). *Other rows:* the first ten entries (the tenth is GPT-6 Astra with direct control) of the public leaderboard [33] as of October 9, 2026, in its order (by Average Score, 33 entries). Gen. is the mean of the standard and randomized halves; WAM: world-action model. Bold: the highest of a column.

Rank	Entry	Type	Average	Gen.	Prec.	Long-Hor.	Memory	Open
–	**PyRUA-Lean** (ours, GPT-6 Astra)	Agent	**58.20 / 51.62**	**50.65 / 45.83**	**47.02 / 37.25**	**63.26 / 48.00**	**63.00 / 63.00**	**67.09 / 64.00**
1	VPP2 [12]	WAM	39.26 / 32.26	36.38 / 29.50	43.03 / 35.17	56.28 / 37.00	56.01 / 55.56	4.61 / 4.08
2	PhysicalRSI [22]	Agent + VLA	36.27 / 31.38	21.30 / 15.62	38.05 / 32.50	46.37 / 37.33	46.74 / 46.56	28.88 / 24.92
3	Awomo-0.5 [3]	WAM	35.34 / 29.64	30.31 / 23.84	39.65 / 33.50	40.36 / 26.75	42.27 / 41.22	24.09 / 22.92
4	Simate-beta [37]	VLA	33.95 / 27.96	35.09 / 27.95	34.35 / 26.92	57.84 / 43.42	33.33 / 33.00	9.12 / 8.50
5	Rex-M1 Preview [11]	VLA	33.79 / 27.44	34.05 / 26.05	32.82 / 23.83	46.89 / 33.00	48.53 / 48.22	6.67 / 6.08
6	VPP2-Preview [12]	WAM	31.40 / 25.62	31.34 / 24.62	33.75 / 25.58	35.31 / 22.17	50.78 / 50.33	5.83 / 5.42
7	InternW0-Δ [21]	WAM	30.77 / 23.91	30.09 / 22.78	31.98 / 23.25	46.29 / 29.33	34.67 / 34.00	10.84 / 10.17
8	Liber-0 Preview [18]	VLM + WAM	30.74 / 25.52	24.99 / 18.61	38.28 / 33.17	45.98 / 33.17	37.77 / 37.33	6.68 / 5.33
9	Liber-0 Lite [18]	WAM	29.24 / 24.23	25.36 / 19.11	36.60 / 31.08	45.50 / 32.92	35.68 / 35.44	3.06 / 2.58
10	GPT-6 Astra, direct control [47]	Agent	28.97 / 22.48	33.36 / 30.50	12.65 / 4.00	21.45 / 8.25	43.04 / 38.67	34.36 / 31.00

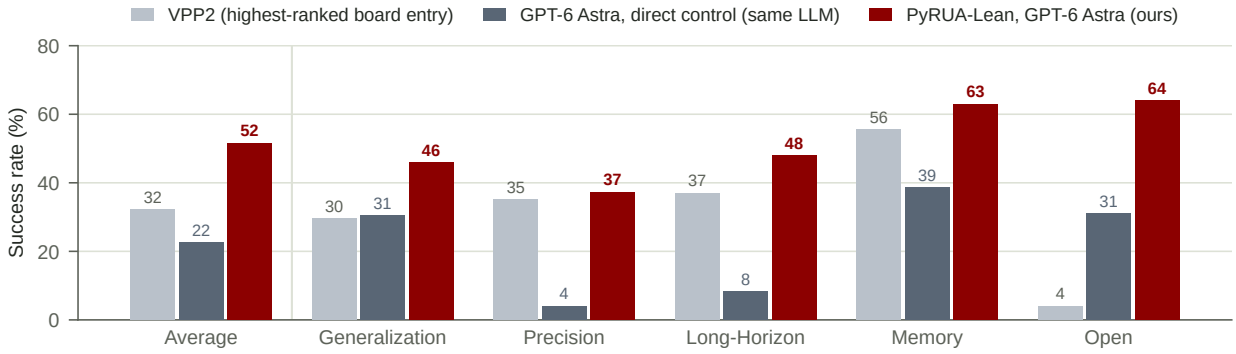


Figure 6 Success rate on RoboDojo, on average and per capability dimension: the highest-ranked entry of the public board, GPT-6 Astra controlling the end-effector directly [47], and the same LLM acting through PyRUA-Lean (ours, one evaluation seed).

systems as a whole, not the observation, which is the same.

Where it fails. Three tasks are solved in every episode (`stack_blocks`, `cover_blocks`, `stack_blocks_by_language`), and six in none (`store_laptop_and_headphones`, `insert_key`, `make_kong`, `swap_blocks`, `press_by_number`, `store_tools_in_toolbox`). The failures are not spread evenly (Table 6): pretrained policies reach 83–96% on pressing buttons a given number of times (`press_by_number`), on a tile game against an opponent (`make_kong`), and on building a tower from blocks and boards (`build_tower`), where our agent is at or near zero. The other tasks without a success ask for a key to be handed over, inserted and turned, for a button to be pressed after each move, or for objects to be stored in stands and fitted compartments. Several of these tasks involve contact whose outcome the agent's feedback does not report, such as a button press, or fits that are tight relative to the accuracy of its perception. We have not analyzed these failures episode by episode.

Cost. An episode takes 25.7 LLM calls on average, 19.3 when it is solved and 32.2 when it is not; the longest took 76, and no episode ran more than 73 code cells, so none reached the budget of 100. An episode consumes 750k input tokens, cached input included, and 9.0k output tokens, and lasts about 12.7 minutes. At about $2.4 per episode at our model gateway, the 2,100 episodes of the seed cost about $5,100. At this cost we evaluated two configurations, this one and the one with depth of Section 4.4, which ran on three seeds; we did not run the evaluation with a VLA policy added to the primitives.

4.4 The Same Agent with Depth

RoboDojo's evaluation client can also add a depth image and the matrices of each camera to the observation. The leaderboard does not admit this depth for agent entries, so the results above do not use it and this configuration is not compared with the board. With depth, `locate` measures an object from the depth pixels under its mask, `back_project` reads the depth at a pixel, and `camera_meta` returns the matrices the client sends;

Table 6 Tasks with the largest differences between PyRUA-Lean and the best entry of the public board on that task (SR in %; the board gives per-task results for 57 policies and ranks 33). Appendix K has all 42 tasks.

Task	Ours	Best	Best board entry
Largest margins over the best board entry			
`imitate_sorting_sequence`	98.0	36.0	GPT-6 Astra (direct)
`classify_objects_by_language`	92.0	30.0	GPT-6 Astra (direct)
`stack_blocks_by_language`	100.0	40.0	GPT-6 Astra (direct)
`sort_nesting_dolls_by_size`	74.0	18.7	VPP2
`play_stacking_toy`	66.0	23.3	VPP2
`classify_objects`	98.0	56.0	GPT-6 Astra (direct)
Largest deficits to the best board entry			
`press_by_number`	0.0	96.0	VPP2
`make_kong`	0.0	90.0	GalaxeaVLA (G0.5)
`build_tower`	2.0	83.3	Simate-beta
`pour_liquid_into_cup`	8.0	67.3	Rex-M1 Preview
`insert_key`	0.0	39.3	PhysicalRSI
`swap_blocks`	0.0	38.7	Simate-beta

Table 7 PyRUA-Lean on RoboDojo from RGB images alone, and with the simulator's depth added to the observation: each value Score / SR (%), as in Table 5. The configuration with depth was evaluated on three seeds; the standard deviation over them is the population one, as in RoboDojo's summary. The configuration with depth is not compared with the leaderboard.

Observation	Episodes	Average	Gen.	Prec.	Long-Hor.	Memory	Open
The same evaluation seed							
RGB (Section 4.3)	2,100	58.20 / 51.62	50.65 / 45.83	47.02 / 37.25	63.26 / 48.00	63.00 / 63.00	67.09 / 64.00
RGB and depth	2,100	58.59 / 52.42	49.71 / 43.83	49.92 / 41.00	63.02 / 49.00	64.00 / 64.00	66.29 / 64.25
Three evaluation seeds							
RGB and depth, mean	6,300	58.81 / 52.91	51.11 / 45.44	49.78 / 41.08	64.73 / 51.75	64.02 / 64.00	64.39 / 62.25
standard deviation over the seeds		0.15 / 0.35	1.10 / 1.23	0.57 / 0.51	1.39 / 2.15	0.02 / 0.00	2.16 / 2.16

the notes on measuring with the wrist cameras are dropped, and nothing else changes (Appendix I).

Depth helps: on the seed that both configurations ran, it raises the Average SR from 51.6% to 52.4% and the Score from 58.2 to 58.6, most on Precision (37.3% to 41.0%; Table 7). Over three evaluation seeds (6,300 episodes), the configuration with depth reaches an Average SR of 52.9% and a Score of 58.8. The seeds, which have different layouts, give 52.4%, 53.2% and 53.1%, a standard deviation of 0.35 points: one seed is a close estimate of the success rate, and the lead over the leaderboard in Section 4.3 is not an effect of the seed.

5 PyRUA-Lean Coder: Training an Open Model to Use the Interface

Sections 3 and 4 use a frontier LLM behind an API. This section asks whether a small open-weight model can learn to act through the same interface. We train the planner itself: the LLM that reads the task, writes a cell, reads its output and the images it asked for, and decides what to do next. The primitives, including the frozen VLA policies, stay as in Section 3. We call the trained planner **PyRUA-Lean Coder v0.1-4B**.

5.1 Training

Model. We start from Qwen3.5-4B [30], an open-weight vision-language model with 4B parameters that reasons before it answers.

Episodes and reward. A training episode is the interaction of Section 3: the prompt with the API reference, the operating guides and the task card (instruction, initial robot state, first camera images), followed by alternating LLM calls (the model reasons, then calls `python`) and tool results (text, and the images the code requested). The reward is 1 if the benchmark's success check passes and 0 otherwise, with no shaping and no penalty for LLM calls or simulator steps. To keep rollouts short, a training episode is limited to 24 code cells and 25 minutes, against 40 cells and 90 minutes when the same benchmark is evaluated, and an LLM call to

Table 8 Success rate (%) of Qwen3.5-4B before and after training (PyRUA-Lean Coder v0.1-4B), and of Qwen3.5-27B, on the 700 task instances of Section 3, one episode per instance. Training used 45 LIBERO-PRO instances only. *Last row:* GPT-6 Astra with PyRUA-Lean from Table 2, with its LIBERO-PRO result split in the same way, run in a different agent loop (Section 5.2), for reference. Bold: the highest value of a column among the first three rows. RC365: RoboCasa365.

Planner	Trained on	Not seen in training					
	LIBERO-PRO 45 instances	LIBERO-PRO other tasks (155)	RoboTwin 2.0 (250)	RC365 atomic (90)	RC365 composite (160)	All unseen (655)	All (700)
Qwen3.5-4B	37.8	13.5	14.0	52.2	12.5	18.8	20.0
Qwen3.5-27B	64.4	**38.1**	20.8	65.6	13.1	29.2	31.4
PyRUA-Lean Coder v0.1-4B	**71.1**	37.4	**52.4**	**76.7**	**19.4**	**44.1**	**45.9**
GPT-6 Astra (Section 3)	100.0	92.3	69.2	86.7	39.4	69.8	71.7

4,096 new tokens.

Training instances. We train on 45 LIBERO-PRO task instances: three tasks from each of the goal, spatial, and object suites at five seeds. The other 31 LIBERO-PRO tasks, among them the whole long-horizon suite, and all of RoboTwin 2.0 and RoboCasa365 are never used.

Algorithm. We use group relative policy optimization (GRPO) [35], a reinforcement-learning algorithm. In each iteration, 16 instances are drawn and the current model plays each of them 8 times, 128 episodes in all. An episode's advantage is its reward normalized within its group of 8, and the clipped policy-gradient loss (clip range 0.2 below and 0.28 above, as in [46]) is applied to every token the model generated in the episode, reasoning and code alike. An iteration makes two optimizer steps of 64 episodes each, with Adam at a constant learning rate of 10^{-6} and without a KL or an entropy term. The model is fine-tuned in full, without adapters, starting from the base weights: there is no supervised stage and no demonstration data. A group that loses an episode to an infrastructure failure is dropped and sampled again. Training runs for 30 iterations, that is 3,840 episodes and 60 optimizer steps, and the checkpoint after the last iteration is the one we evaluate and will release (Appendix J lists all settings).

Infrastructure. Training uses the miles framework [32] with SGLang [49] serving the rollouts, on one node with eight GPUs; the simulators and the frozen VLA and SAM 3 services run next to it. The run takes about twelve hours.

5.2 Evaluation Protocol

We evaluate on the 700 task instances of Section 3, one episode per instance, with that section's prompt, tools, primitives, frozen VLA policies and budgets. A budget counts the code cells of an episode, 40 or, on RoboCasa365 composite tasks, 100; GPT-6 Astra writes one cell per LLM call, which makes it the LLM-call budget of Section 3. The runs differ from Section 3's GPT-6 Astra runs in the agent loop, which is PyRUA-Lean's own instead of the Codex CLI and requires a tool call in every LLM call; in the context, which is 65,536 tokens (an episode that exceeds it ends there); and in the wall-clock limit, which is 90 minutes. However an episode ends, the benchmark's success check decides whether it is solved. Sampling follows the model card's settings for reasoning (temperature 1.0, top-p 0.95, top-k 20), with at most 8,192 new tokens per LLM call. The base model, the trained model and Qwen3.5-27B [31] are evaluated identically.

LIBERO-PRO's 200 instances contain the 45 training instances, so we report them separately from the 155 instances of tasks the model was not trained on. Together with RoboTwin 2.0 and RoboCasa365, these give 655 instances that training never saw. An instance whose episode failed on infrastructure is run again and does not count until it has a result.

5.3 Results

GRPO on 45 instances lifts the 4B model on every benchmark. On the training instances, Qwen3.5-4B goes from 37.8% to 71.1% (Table 8). On LIBERO-PRO tasks it was not trained on, it goes from 13.5% to 37.4%; on RoboTwin 2.0 from 14.0% to 52.4%; on RoboCasa365 atomic tasks from 52.2% to 76.7%; and on composite tasks from 12.5% to 19.4%. Over all instances not seen in training, the success rate rises from 18.8% to 44.1%

Table 9 How episodes run and end, on the instances not seen in training (means, except code cells per LLM call, a median). *Code cells* counts the cells a model wrote, including those refused once its budget was spent. *Images requested* leaves out the two or three pictures of the task card, which every episode starts with. *Gave up*: the model ended the episode itself without success; *Cell budget*, *Wall clock*, *Context overflow*: the episode was stopped at that limit without success. Every episode is in exactly one of the five columns on the right. An episode that reached a limit with its task already complete counts as solved (three episodes of Qwen3.5-4B and one episode of Qwen3.5-27B).

Planner	Per episode				How the episode ended (%)				
	LLM calls	Code cells per LLM call	Code cells	Images requested	Solved	Gave up	Cell budget	Wall clock	Context overflow
Qwen3.5-4B	9.4	9.0	89.4	19.3	18.8	6.9	52.7	2.4	19.2
Qwen3.5-27B	35.5	1.0	34.9	22.5	29.2	4.6	35.0	0.3	31.0
PyRUA-Lean Coder v0.1-4B	21.1	1.0	21.0	0.8	44.1	25.2	10.5	16.8	3.4



Figure 7 Training of PyRUA-Lean Coder v0.1-4B: success rate of the rollouts in each GRPO iteration (16 training instances × 8 episodes, under the training limit of 24 code cells).

(Figure 1(e)). The trained 4B model is thereby above Qwen3.5-27B, which reaches 29.2% on these instances under the same protocol.

The gain transfers across simulators, robots, and VLA policies. RoboTwin 2.0 is a dual-arm benchmark in another simulator, with LingBot-VLA as its policy primitive; RoboCasa365 has a mobile manipulator in kitchens, with RLDX-1. Neither appears in training, which saw one single-arm robot and $\pi_{0.5}$ on nine tabletop tasks. The transfer is therefore not explained by seeing the evaluation tasks, scenes, or robots.

What changes is how the model uses the interface. Table 9 shows the three models on the unseen instances. The base model writes 9.0 code cells per LLM call (median over its episodes; more than 1.5 in 89% of them): it commits a batch of actions before seeing the result of the first. Its episodes last 9.4 LLM calls on average; 52.7% of them end on the budget of code cells and 19.2% on a context overflow. Qwen3.5-27B writes one cell per LLM call, but it requests 22.5 images per episode and ends 35.0% of its episodes on the cell budget and 31.0% on a context overflow. After training, the 4B model writes one cell per LLM call and reads its result before the next. It requests 0.8 images per episode instead of 19.3, and endings on the cell budget and on a context overflow fall to 10.5% and 3.4%. In exchange, more episodes end because the model gives up (25.2%) or runs into the wall clock (16.8%). This is the feedback-driven use of the interface described in Section 3.2, which the frontier model shows without training; it would also explain why a gain learned on nine tasks carries over to other benchmarks. We do not claim that training improved manipulation skill as such: grasps and motions are still executed by the same primitives and frozen VLA policies.

Training dynamics. The success rate of the training rollouts rises from 0.30 in the first iteration to 0.70 over the last five (Figure 7). These rollouts run on the 45 training instances and under the shorter training limits, so they are not comparable with the evaluation numbers above.

Distance to the frontier model. PyRUA-Lean Coder v0.1-4B solves 45.9% of the 700 instances, against 71.7% for GPT-6 Astra with the same interface and primitives in Section 3. The two numbers come from different agent loops and limits and are not a controlled comparison, but the gap is large. It is smallest on RoboCasa365 atomic tasks and RoboTwin 2.0 and largest on unseen LIBERO-PRO tasks (Table 8).

6 Limitations and Future Work

The interface comparison. The comparison of Section 3 is limited to GPT-6 Astra, RPent's robot stacks and primitive libraries, and simulation, with one run per agent per task instance. Generalization to other planners and primitive configurations, variability across repeated runs, and physical-robot performance remain untested. The comparison evaluates the complete interfaces without fully isolating the effects of primitive composition, persistent state, and selective feedback.

RoboDojo. The RoboDojo agent is a system around a frontier LLM behind an API, with a segmentation model, where most leaderboard entries are single pretrained policies, and we have not measured how much of its margin over direct control comes from the perception primitive and how much from programs. Its failures concentrate on tasks that involve contact it gets no feedback on, or tight fits, and we have not analyzed them episode by episode. We left VLA policies out of the agent because of their run-to-run variance (Section 4.2). An episode costs dollars and minutes, which is why we evaluated only two configurations, with and without depth, and have not measured what a VLA policy would add as a further primitive.

PyRUA-Lean Coder. PyRUA-Lean Coder v0.1-4B is a first version: one model size and one training run, on 45 instances of one benchmark and with success as the only reward. What the model demonstrably learned is to use the interface step by step, and it remains far below the frontier model on the same tasks. We evaluated it with the primitives and VLA policies of Section 3 only; whether it can drive the VLA-free primitives of Section 4 is untested.

Future work will address these limitations, measure execution latency and recovery on physical robots, examine how primitive granularity and cross-episode reuse of validated routines affect success, inference cost, and adaptability, and train larger models, on more benchmarks and for longer.

7 Conclusion

We presented PyRUA-Lean, an interactive code-execution interface that composes existing robot primitives, including VLA policies, and controls which execution feedback enters the LLM context. Across 700 simulated task instances, PyRUA-Lean outperforms a tool-calling baseline using the same GPT-6 Astra planner and underlying primitives under equal LLM-call budgets, increasing overall success from 63.1% to 71.7%, and on instances solved by both agents it uses 49% fewer LLM calls and 65% fewer input tokens. With a handful of geometric primitives, no learned policy acting on the robot, and no privileged information, the same planner reaches 51.6% average success rate on RoboDojo, above every entry of the public leaderboard. And the interface can be learned: GRPO on 45 task instances turns an open 4B model into a planner that solves 44.1% instead of 18.8% of the instances it never trained on, across three benchmarks. These results show the value of programmatic primitive composition for frontier and small models alike, and motivate further study of feedback management and of training in robot agents.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.

[2] Anthropic. Code execution with MCP: Building more efficient agents. Anthropic Engineering Blog, https://www.anthropic.com/engineering/code-execution-with-mcp, 2025.

[3] Awomo-WAM Team. Awomo-0.5: World action model pre-trained at scale. https://github.com/Awomo-WestlakeDI/Awomo-0.5, 2026.

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, et al. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025.

[5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.

[6] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, et al. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.

[7] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[8] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen, Songling Liu, Haowen Yan, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.

[9] Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, et al. Show-Harness: Just a VLM agent can play robots. *arXiv preprint arXiv:2609.10522*, 2026.

[10] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, et al. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. In *International Conference on Machine Learning (ICML)*, 2026.

[11] Futian Lab and VisIncept. Rex-M1-preview: Policy adapter for RoboDojo in XPolicyLab. https://github.com/XPolicyLab/XPolicyLab/tree/main/policy/Rex_M1_preview, 2026.

[12] Yanjiang Guo, Haodong Yan, Zhide Zhong, Zhongru Zhang, Qingyuan Yang, Qingzhou Lu, Xiaoyu Chen, Yen-Jen Wang, Shuying Deng, Chenghan Yang, Puzhen Yuan, Chenxin Liu, Tun Ban, Xiang Zhu, Yichen Liu, Kun Feng, Haoang Li, and Jianyu Chen. Video prediction policy 2: Predict better, act better. *arXiv preprint arXiv:2610.10270*, 2026.

[13] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2022.

[14] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.

[15] Dongyoung Kim, Huiwon Jang, Myungkyu Koo, Suhyeok Jang, Taeyoung Kim, Beomjun Kim, Byungjun Yoon, Changsung Jang, et al. RLDX-1 technical report. *arXiv preprint arXiv:2605.03269*, 2026.

[16] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.

[17] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[18] LiberAI. Liber-0 Lite: RoboDojo evaluation checkpoint. https://huggingface.co/LiberAI/Liber0-Lite-Robodojo, 2026.

[19] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[20] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, et al. ASPIRE: Agentic /skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[21] Xingyu Miao, Zizun Li, Baole Fang, Kaiwen Song, Tenghui Wang, Hanxue Zhang, Yating Wang, Xudong Li, Yuping He, Xueyuan Wei, et al. InternW0-Δ: A world action model bridging predictive dynamics and actions with 20K+ hours of open data. *arXiv preprint arXiv:2609.31394*, 2026.

[22] MMLab & PhysicalRSI contributors. Physical RSI 1.0: Recursive self-harness for scaling embodied skills. Project website and open-source software, https://mmlab.hk/research/PhysicalRSI, 2026. Preview release.

[23] Dhia Naouali, Minghan Wu, Claudia Wong, Abhinav Puthran, and Omar G. Younis. VLCP: Vision language control policy closed-loop code replanning for robot manipulation. *arXiv preprint arXiv:2608.16978*, 2026.

[24] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.

[25] Noematrix Team. RoboRSI: Stable, efficient, and reusable robot self-evolution in complex real-world environments. Blog post, https://lab.noematrix.ai/blog/2-roborsi/, 2026.

[26] OpenAI. Codex CLI. https://github.com/openai/codex, 2025.

[27] OpenAI. GPT-6 Astra: A new generation of intelligence. https://openai.com/index/gpt-6-astra/, 2026.

[28] OpenAI. Pricing. OpenAI API documentation, https://developers.openai.com/api/docs/pricing, 2026.

[29] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[30] Qwen Team. Qwen3.5-4B model card. https://huggingface.co/Qwen/Qwen3.5-4B, 2026.

[31] Qwen Team. Qwen3.5-27B model card. https://huggingface.co/Qwen/Qwen3.5-27B, 2026.

[32] RadixArk. miles: A reinforcement-learning framework for large language models. https://github.com/radixark/miles, 2026.

[33] RoboDojo Team. RoboDojo sim leaderboard. https://robodojo-benchmark.com, 2026. Board dated 2026-10-09, accessed 2026-10-12.

[34] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[35] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[36] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, et al. ProgPrompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[37] SIPAILab. Simate-beta: SIPAI model weights for RoboDojo evaluation. https://huggingface.co/SIPAILab/sipai-robodojo-eval-ae, 2026.

[38] Jiayi Su, Yixin Zheng, Mi Yan, Li Yi, Zhizheng Zhang, and He Wang. GPT 6 Astra as an embodied policy. Technical report, https://anonymous-report-421.github.io/public-website/, 2026.

[39] Kenton Varda and Sunil Pai. Code Mode: the better way to use MCP. Cloudflare Blog, https://blog.cloudflare.com/code-mode/, 2025.

[40] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[41] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024.

[42] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, et al. A pragmatic VLA foundation model. *arXiv preprint arXiv:2601.18692*, 2026.

[43] XPolicyLab Community, Tianxing Chen, Yue Chen, Tian Nian, Zijian Cai, Guangyu Chen, Wenwei Lin, Qiwei Liang, Zanxin Chen, Peicheng Xiang, et al. XPolicyLab: A unified standard and open ecosystem for robot policy evaluation and deployment. *arXiv preprint arXiv:2608.09892*, 2026.

[44] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[46] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[47] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, Yue Chen, and Tianxing Chen. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. *arXiv preprint arXiv:2609.24170*, 2026.

[48] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaxing Qiu, et al. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.

[49] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.

[50] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

Fewer Tokens, Better Action: GPT-6 Astra Robot Agents with 14% Higher Success Rate but 65% Fewer Tokens

Appendix

A The episode of Figure 2, LLM call by LLM call

Figure 2 shows one stretch of an episode. Figure 8 and Table 10 follow the whole episode with each agent: it is an instance both agents solved, close to the median cost of each on LIBERO-PRO (spatial swap, task 7, seed 2). The tool-calling agent reads the guides, localizes, and then moves one step per LLM call, getting three camera images back after every move; it solves the task in LLM call 17, after 33 images in all. The code agent localizes both objects in its first cell, approaches and grasps in its second, closes the gripper and lifts the bowl in its third, checks the grasp in its fourth, and carries and places the bowl in its fifth and sixth; it solves the task in LLM call 6 and asks for 5 images in all. Appendix B follows further episodes in this way, and Appendix C describes what the code agent's cells do.

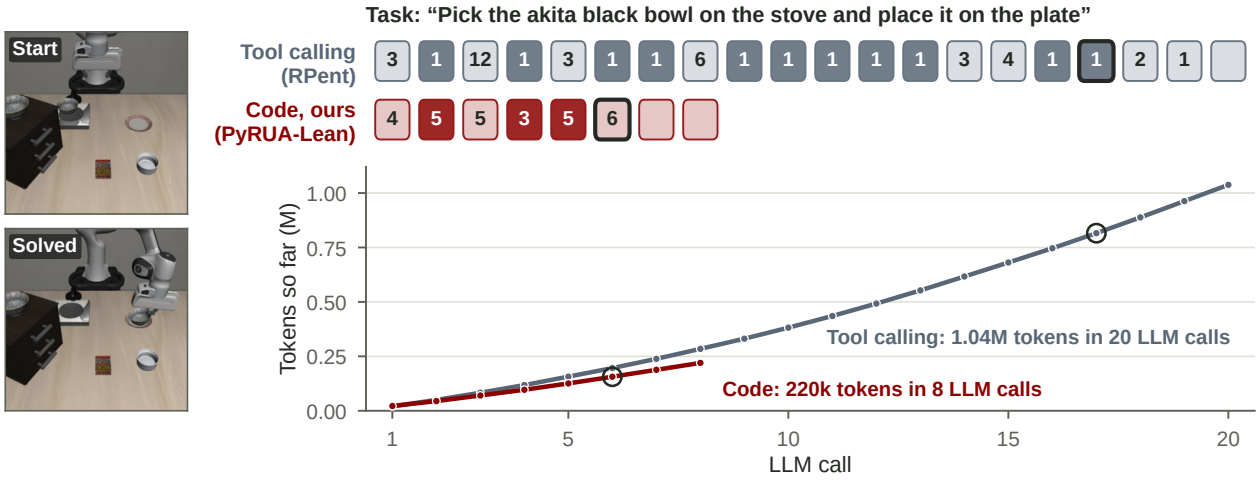


Figure 8 The episode of Figure 2, whole. *Top:* each box is one LLM call; the number is how many tools (tool calling) or primitives (code) it called, dark boxes returned camera images, and the outlined box solved the task. *Bottom:* token consumption so far; circles mark the LLM call that solved the task, reached after 816k tokens with tool calling and 156k with code. *Left:* the agent-view camera at the start and when the task was solved.

Cells 1–6 of the code agent are reproduced exactly as generated by the model.

LLM call 1

```
print(robo.state())
bowl = robo.segment('the black patterned bowl on the stove')
plate = robo.segment('the white plate with red rings')
print('bowl', bowl)
print('plate', plate)
print('table', robo.back_project(630, 610))
```

LLM call 2

```
assert bowl.found and plate.found
approach = robo.move_to([bowl.world_xyz[0], bowl.world_xyz[1], bowl.world_xyz[2]+0.19], gripper=-1)
print('approach', approach)
if approach.reached:
    pick = robo.pi0_pick('pick up the black bowl on the stove', max_chunks=20)
    print('pick', pick)
print(robo.state())
robo.show('agentview')
robo.show('wrist')
```

Table 10 The two episodes of Figure 8, LLM call by LLM call: what each call ran, and the token consumption so far. A check mark is the call that solved the task. Cached input is included, as everywhere.

Call	Tool calling (RPent): tools called	Tokens	Code (PyRUA-Lean): what the cell ran	Tokens
1	`read_text_file` ×3	20k	4 primitive calls: `back_project`, `segment`, `view_env_state`	22k
2	`view_env_state`; 3 images back	50k	5 primitive calls: `move_to`, `pi0_pick`, `view_env_state`; shows agentview and wrist	44k
3	`back_project` ×12	83k	5 primitive calls: `move_to`, `segment`, `set_gripper`, `view_env_state`	70k
4	`move_to`; 3 images back	118k	3 primitive calls: `segment`; shows agentview and wrist	96k
5	`back_project` ×3	157k	5 primitive calls: `move_to`, `segment`, `view_env_state`; shows agentview	126k
6	`move_to`; 3 images back	196k	6 primitive calls: `move_to`, `release`, `segment`, `view_env_state` ✓	156k
7	`move_to`; 3 images back	239k	`finish`	188k
8	`back_project` ×6	284k	final message	220k
9	`pi0_pick`; 3 images back	331k		
10	`set_gripper`; 3 images back	382k		
11	`move_to`; 3 images back	435k		
12	`move_to`; 3 images back	492k		
13	`move_to`; 3 images back	553k		
14	`back_project` ×3	616k		
15	`back_project` ×4	681k		
16	`move_to`; 3 images back	746k		
17	`release`; 3 images back ✓	816k		
18	`write_text_file` ×2	888k		
19	`finish`	963k		
20	final message	1.04M		

LLM call 3

```
print(robo.set_gripper(+1, steps=8))
held_state = robo.state()
print('lift', robo.move_to([held_state.eef_pos[0], held_state.eef_pos[1], 1.18], gripper=+1, step_clip=0.012))
held_bowl = robo.segment('the black patterned bowl held by the gripper')
print('held bowl', held_bowl)
print(robo.state())
```

LLM call 4

```
robo.show('agentview')
robo.show('wrist')
print('bowl now', robo.segment('the black patterned bowl on the stove'))
```

LLM call 5

```
held_bowl = robo.segment(point=(267, 339))
print('held localization', held_bowl)
assert held_bowl.found and held_bowl.world_xyz[2] > 1.05
carry_state = robo.state()
grip_offset = np.array(carry_state.eef_pos[:2]) - np.array(held_bowl.world_xyz[:2])
place_xy = np.array(plate.world_xyz[:2]) + grip_offset
print('grip offset', grip_offset, 'place xy', place_xy)
mid_xy = (np.array(carry_state.eef_pos[:2]) + place_xy)/2
for waypoint_xy in [mid_xy, place_xy]:
    move = robo.move_to([*waypoint_xy, carry_state.eef_pos[2]], gripper=+1, step_clip=0.012)
    print('carry', move)
    if not move.reached or move.terminated:
        break
robo.show('agentview')
```

LLM call 6

```
held_at_plate = robo.segment(point=(314, 795))
print('bowl above plate', held_at_plate)
assert held_at_plate.found
correction = np.array(plate.world_xyz[:2]) - np.array(held_at_plate.world_xyz[:2])
place_xy = np.array(robo.state().eef_pos[:2]) + correction
surface_map = robo.world_map()
bowl_patch = surface_map[290:365, 740:850]
bowl_heights = bowl_patch[:,:,2]
bowl_heights = bowl_heights[(bowl_heights>1.04) & (bowl_heights<1.17)]
bottom_z = float(np.quantile(bowl_heights, 0.02))
placement_z = plate.world_xyz[2] + robo.state().eef_pos[2] - bottom_z + 0.007
print('correction', correction, 'bottom', bottom_z, 'placement z', placement_z)
assert np.linalg.norm(correction)<0.07 and 0.95<placement_z<1.05
lower = robo.move_to([*place_xy, placement_z], gripper=+1, step_clip=0.012, tol=0.005, max_steps=100)
print('lower', lower)
if lower.reached and not robo.done:
    print('release', robo.release())
print('done', robo.done)
if not robo.done:
    robo.show('agentview')
```

B More episodes

Table 11 and Figure 9 add five tasks per benchmark that both agents solved: on each benchmark, the task instances whose token factor (tool calling's token consumption over code's, whole episodes) is closest to the benchmark's median, at most one per task and none of Figure 8's task.

Table 11 The episodes of Figure 9: LLM calls and token consumption of each whole episode, with tool calling (*Tool*) and with code (*Code*); shaded: how many times fewer tokens code consumed.

Benchmark	Task	Seed	LLM calls		Tokens		
			Tool	*Code*	*Tool*	*Code*	×
LIBERO-PRO	object swap 0: Pick the alphabet soup and place it in the basket	1	21	8	1.09M	219k	5.0×
	spatial swap 5: Pick the akita black bowl on the ramekin and place it on the plate	1	18	7	918k	188k	4.9×
	goal swap 4: Put the bowl on the top of the drawer	3	19	7	991k	199k	5.0×
	spatial swap 9: Pick the akita black bowl on the wooden cabinet and place it on the plate	3	24	9	1.43M	288k	5.0×
	object swap 3: Pick the bbq sauce and place it in the basket	1	17	6	814k	161k	5.0×
RoboTwin 2.0	place container plate	100000	7	4	151k	89k	1.7×
	open laptop	100008	7	4	149k	88k	1.7×
	place empty cup	100001	7	4	150k	89k	1.7×
	rotate qrcode	100001	7	4	151k	89k	1.7×
	move pillbottle pad	100002	7	4	152k	91k	1.7×
RoboCasa365 atomic	CloseToasterOvenDoor	1	5	3	91k	74k	1.2×
	PickPlaceToasterToCounter	4	5	3	92k	75k	1.2×
	CoffeeSetupMug	5	5	3	92k	75k	1.2×
	TurnOnSinkFaucet	4	5	3	91k	74k	1.2×
	TurnOnElectricKettle	4	5	3	91k	75k	1.2×
RoboCasa365 composite	ScrubCuttingBoard	3	8	4	159k	101k	1.6×
	WashLettuce	1	8	4	162k	100k	1.6×
	StackBowlsCabinet	1	6	3	117k	78k	1.5×
	CuttingToolSelection	2	18	9	438k	253k	1.7×
	LoadDishwasher	5	15	7	361k	193k	1.9×

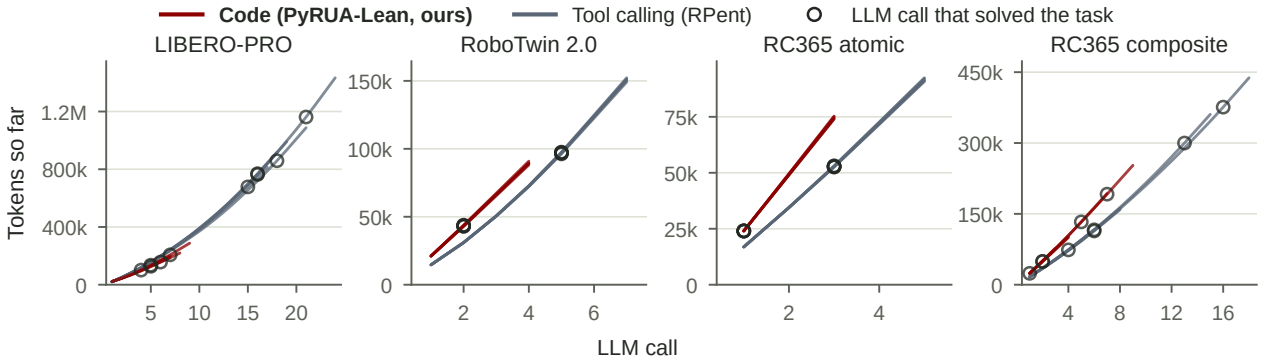


Figure 9 Token consumption so far of the episodes in Table 11, five per benchmark, LLM call by LLM call; circles mark the solving call. On RoboTwin 2.0 and RoboCasa365's atomic tasks the median episodes of both agents are nearly identical from task to task, so their lines overlap.

C What the code agent writes

We read the code agent's cells on every benchmark, in solved and failed episodes, with and without the VLA policy, and then counted what they do over all 700 episodes of the main comparison, 11,145 cells, each episode cut at its budget (Table 12).

A cell acts more than a tool call does. A code cell runs 2.1 acting primitives on average (moves, grasps, gripper commands and VLA calls), and 45% of cells run two or more; an LLM call of tool calling runs 0.65. Of tool calling's LLM calls that call a tool, 89% call exactly one; of the 1,907 that call several, 1,441 batch back-projections and 292 read or write files, and only 45 of all 18,585 run two acting tools together. Five habits of the code agent make up the difference, and none of them fits in one tool call: the first two fold steps that tool calling spreads over several LLM calls into one, the next two build what the tool list does not offer, and the last decides when to look.

Guarded chains (19% of cells, 45% on LIBERO-PRO). A cell runs several primitives in a row and makes each depend on the result of the one before (Listing 1): it moves above the bowl, grasps with the VLA policy only if the move arrived and the task is not done, and looks only if it is still not done. With tool calling, each link of such a chain is an LLM call.

Retries (8% of cells, 18% on RoboCasa365's atomic tasks). Where a VLA policy may stop short of the goal, the cell runs it again until the task reports success (Listing 2). With tool calling, every retry is another LLM call, which brings back three camera images.

Perception of its own (30% of cells compute with NumPy, for perception or geometry). A cell can read a camera's image and point cloud as arrays and locate objects itself: a color mask for a block, the points above a height for its top, the principal axis of those points for its orientation, and from that the yaw of the grasp (Listing 3). A tool-calling agent gets what its tools compute (images, the 3D point at a pixel it names, statistics of a box it names), and must work out anything else in its reasoning.

Skills of its own (2% of cells define a helper function, 7% call one that an earlier cell defined). In Listing 4, the agent writes a descent that moves down in 8 mm steps and stops as soon as a step does not reach its target, stops going down, or drifts sideways, and it reuses the function in its next cell. Helpers like this are how the code agent grasps without a VLA policy (Section 3.7).

Looking when needed (20% of cells). The code agent looks often, 84% of its cells ask for a camera image, but it chooses when and through which camera: 20% of its cells look only if the task is not yet done, where tool calling gets three images after every move whether it needs them or not.

And single steps (35% of cells, 54% on RoboTwin 2.0). Many cells take one step and look, like a tool call. On RoboTwin 2.0 the agent often runs the VLA policy one chunk at a time and checks the result after each; code saves the fewest LLM calls there (Table 2).

Table 12 What the code agent's cells do, over all 700 episodes of the main comparison (each cut at its budget), against what an LLM call of tool calling does on the same task instances. Acting primitives are moves, grasps, gripper commands and VLA calls. RC365: RoboCasa365.

	All	LIBERO-PRO	RoboTwin 2.0	RC365 atomic	RC365 composite
Acting primitives per code cell	2.1	2.0	1.9	2.1	2.4
Acting tools per LLM call of tool calling	0.65	0.60	0.69	0.56	0.67
Share of code cells that					
run two or more acting primitives	45%	61%	24%	50%	56%
take a single step and look	35%	14%	54%	33%	28%
make an action depend on an earlier result	19%	45%	12%	20%	14%
loop over motions	14%	6%	12%	22%	17%
retry a VLA policy	8%	0%	0%	18%	17%
compute with NumPy	30%	25%	31%	29%	30%
reuse a variable from an earlier cell	19%	33%	18%	10%	14%
call a helper function from an earlier cell	7%	7%	6%	2%	9%
ask for a camera image	84%	82%	78%	90%	90%
look only if the task is not done	20%	19%	8%	58%	24%

```
assert bowl.found and plate.found
bowl_xy = np.array(bowl.world_xyz[:2])
plate_xyz = np.array(plate.world_xyz)
table_z = robo.back_project(837,660).world_xyz[2]
prepose = [float(bowl_xy[0]),float(bowl_xy[1]),table_z+0.22]
approach = robo.move_to(prepose,gripper=robo.OPEN)
print('APPROACH',approach)
if approach.reached and not robo.done:
    picked = robo.pi0_pick('pick up the black bowl between the plate and the ramekin', max_chunks=20)
    print('PICK',picked)
print('STATE',robo.state())
if not robo.done:
    robo.show('agentview')
    robo.show('wrist')
```

Listing 1 A guarded chain: cell 2 of the code agent in LIBERO-PRO, spatial swap, task 0, seed 0, exactly as the model wrote it.

D Instances only one agent solved

Code alone solves 96 task instances of the main comparison, and tool calling alone 36. In 52 of the 96, tool calling ran out of LLM calls; in the other 44, it ended the episode itself and reported failure. We read both episodes of each of those 44. In about a quarter, the code agent only called the VLA policy, and the policy succeeded in its episode but not in tool calling's. In most of the others, both agents met the same setback, most often a VLA grasp that missed or an object that fell. Tool calling then asked the policy again, or stopped with most of its budget left; the code agent closed the gripper itself, scripted the recovery from the same primitives, checked each step, and in some episodes computed from the camera's point cloud what no tool returns. Figures 10 and 11 follow two instances, one where tool calling ended the episode itself and one where it ran out of LLM calls.

E Cost accounting

Episode-level accounting. We removed memory-, recipe-, and audit-related instructions from the prompts and operating guides, while retaining the baseline's original tool descriptions. These descriptions may still prompt the baseline to save episode-local recipe and audit files after success. Such files are stored in the episode's own output directory, and inspection of the evaluation traces found no cross-episode file reads. Any associated post-success LLM calls are included in the reported whole-episode metrics.

Prices. Every LLM call of every episode is billed from its own token usage at GPT-6 Astra's list prices as of September 2026 [28]: \$10 per million input tokens, \$1 for cached input, \$12.50 for cache writes and \$50 for output, reasoning included. An LLM call above 272k input tokens would cost twice as much for input and 1.5 times as much for output; none reached it.

```
print(robo.task)
print(robo.success_criteria())
for attempt in range(3):
    result = robo.rldx_arm()
    print(result)
    print(robo.state())
    if robo.done or result.status != 'cap':
        break
if not robo.done:
    robo.show('agentview')
    robo.show('wrist')
```

Listing 2 A retry: cell 1 of the code agent in RoboCasa365 atomic, CloseToasterOvenDoor, seed 1, exactly as the model wrote it.

```
mask=red&(rows<92)&(cols>98)&(cols<165)&(world[...,2]>0.745)
points=world[mask]
upper=points[points[:,2]>0.794]
mean_xy=np.mean(upper[:,:2],axis=0)
eigenvalues,eigenvectors=np.linalg.eigh(np.cov(upper[:,:2].T))
long_axis=eigenvectors[:,1]
if long_axis[1]<0: long_axis=-long_axis
short_axis=np.array([long_axis[1],-long_axis[0]])
projection_long=upper[:,:2]@long_axis
projection_short=upper[:,:2]@short_axis
block_center=long_axis*((np.min(projection_long)+np.max(projection_long))/2)+short_axis*((np.min(projection_short)+np.max(projection_short))/2)
print('flat centre',block_center,'long axis',long_axis,'extents',np.ptp(projection_long),np.ptp(projection_short))
left_grasp_xy=block_center+0.045*long_axis
right_grasp_xy=block_center-0.05*long_axis
yaw=math.atan2(long_axis[1],long_axis[0])
q_down=np.array([math.cos(yaw/2)/math.sqrt(2),-math.sin(yaw/2)/math.sqrt(2),math.cos(yaw/2)/math.sqrt(2),math.sin(yaw/2)/math.sqrt(2)])
print('grasp',left_grasp_xy,'receive',right_grasp_xy,'quat',q_down)
safe=np.array(robo.state().left.eef_pos);safe[2]=1.03
if checked_move('left',safe,quat=robo.state().left.eef_quat,substeps=20):
    waypoint=np.array([left_grasp_xy[0],0.015,1.03])
    if checked_move('left',waypoint,quat=q_down,substeps=25):
        print(robo.move_to('left',[left_grasp_xy[0],left_grasp_xy[1],0.97],quat=q_down,substeps=25))
print(robo.state().left)
robo.show('head')
```

Listing 3 Perception of its own: cell 5 of the code agent in RoboTwin 2.0 without the VLA policy, handover block, seed 100003, exactly as the model wrote it. `checked_move`, a helper it defined in an earlier cell, moves an arm and reports whether the move arrived.

LLM calls, not seconds. We count LLM calls rather than time episodes, because the latency of an LLM call on a shared model gateway varies too much from one call to the next to compare.

All-episode accounting. The efficiency metrics in Table 2 are computed on jointly solved instances, while success rates use all task instances. Aggregating over all episodes, including failures, the tool-calling-to-PyRUA-Lean input-token ratios are 3.1× on LIBERO-PRO, 1.8× on RoboTwin 2.0, 1.6× on RoboCasa365 composite tasks, and 1.1× on RoboCasa365 atomic tasks; the ratios of the estimated inference costs are 2.5×, 1.6×, 1.4×, and 1.0×. On the atomic tasks an episode costs $0.94 with tool calling and $0.96 with PyRUA-Lean: failed episodes on these tasks tend to approach the limit of 40 LLM calls for both agents, while PyRUA-Lean uses more input tokens per LLM call.

First prompts and output. The median first prompt, which holds the system prompt, the guides and the tool list or, for the code agent, the API reference and a task card with the first camera images, is 20k for tool calling and 22k for code on LIBERO-PRO, 15k and 21k on RoboTwin 2.0, and 17k and 24k on RoboCasa365. On jointly solved instances, output tokens, reasoning included, are at most 17% of either agent's bill on any benchmark.

F Benchmarks and task instances

Table 13 lists what the main comparison ran: 700 task instances and 1,400 episodes. LIBERO-PRO runs every task at seeds 0–4, RoboTwin 2.0 at seeds 100000–100004 and RoboCasa365 at seeds 1–5. Where RoboTwin 2.0 cannot build a task at a seed, we use the next seed it can build, and an episode that ended because its

```
def lower_guarded(arm_name, target_tcp_z):
    initial=robo.state().arm(arm_name)
    target=np.array(initial.eef_pos)
    final_eef_z=target_tcp_z-0.12*initial.approach[2]
    while target[2]>final_eef_z+0.001 and not robo.done:
        before=robo.state().arm(arm_name)
        target[2]=max(final_eef_z,target[2]-0.008)
        result=robo.move_to(arm_name,target,substeps=8)
        after=robo.state().arm(arm_name)
        print('descent',result.reached,tuple(round(v,4) for v in after.tcp_pos))
        if result.terminated or not result.planned or not result.reached or after.eef_pos[2]>=before.eef_pos[2]-0.001 or np.linalg.norm(np.array
      (after.eef_pos[:2])-target[:2])>0.01:
            print('stop',result)
            break
lower_guarded('right',0.800)
robo.show('right_wrist')
robo.show('head')
```

Listing 4 A skill of its own: cell 7 of the code agent in RoboTwin 2.0, place object stand, seed 100002, exactly as the model wrote it.

simulator or VLA server stopped answering was run again; one RoboCasa365 instance (CoffeeSetupMug, seed 4) renders only noise on every camera, so that task runs at seed 6 instead.

Table 13 The task instances of the main comparison. Each instance runs once with each agent.

Benchmark	Tasks	Seeds per task	Task instances	LLM-call budget
LIBERO-PRO	40	5	200	40
RoboTwin 2.0	50	5	250	40
RC365 atomic	18	5	90	40
RC365 composite	32	5	160	100
All	140		700	

G Ablation settings

The ablations of Table 3 run on every task instance of the main comparison, at its seeds. Every setting of a group uses the same task instances, and its first setting reuses the episodes of the main comparison.

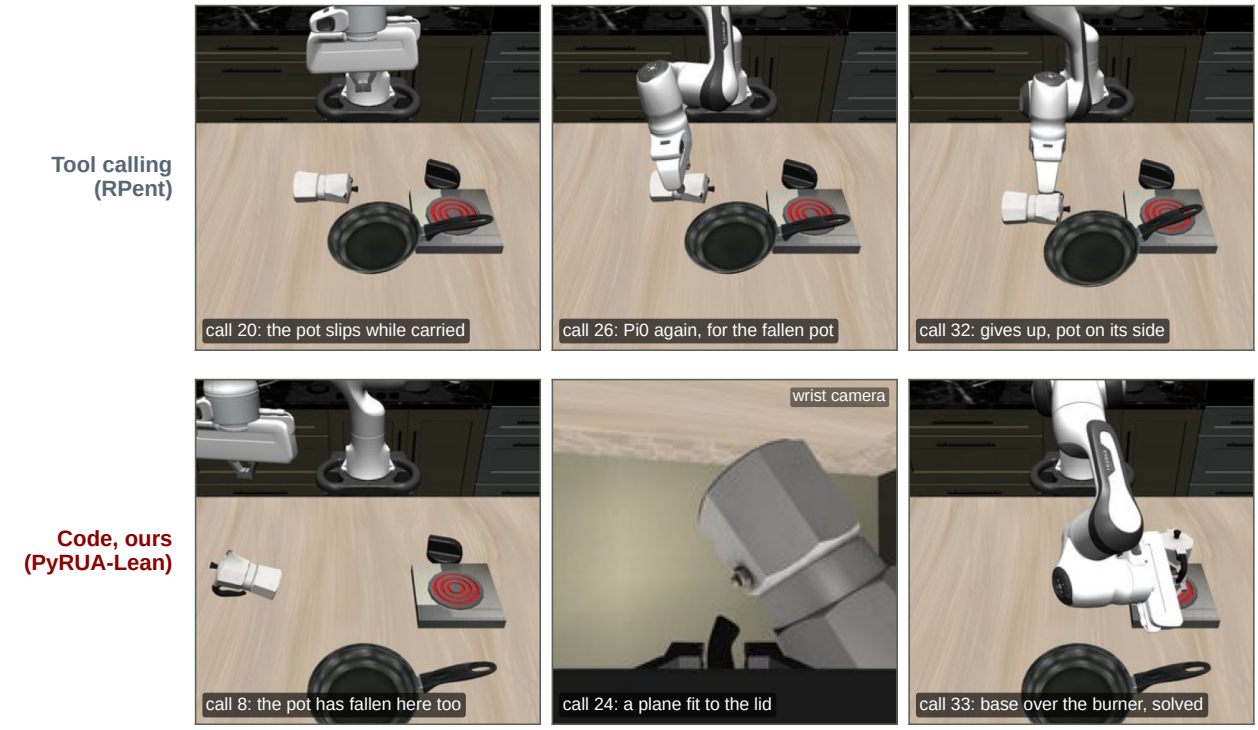
Tool calling with images on demand. On every LIBERO-PRO task instance, the tool-calling agent ran once more with one change: its tools return their text but no camera images, which it sees only when it calls RPent's camera tool (three views per call), and its prompt no longer promises images after every tool call. Code needs no such run, since a PyRUA-Lean cell returns images only when it asks for them; the Code column in Table 14 reuses the PyRUA-Lean episodes from the main comparison.

Table 14 Tool calling with camera images after every move (the LIBERO-PRO baseline of Table 3) and only on demand, against code, on all 200 task instances of LIBERO-PRO. *On the 131 instances all three solved (means).

	Tool, images after every move	*Tool*, images on demand	*Code*
Success rate (%), all task instances	83.0	68.5	94.0
Failures: gave up / LLM-call budget	9 / 25	17 / 46	5 / 7
LLM calls per solved episode*	19.9	22.8	7.4
Input tokens per solved episode*	1.07M	1.07M	222k

H RoboDojo: agent settings and checks

Agent settings. The settings are the same for all tasks and for both configurations: a budget of 100 LLM calls per episode, counted on the calls that run a code cell, the task's own step limit, and a wall-clock limit of 90 minutes, which no episode reached; images only when the code asks for them; at most 8,192 output tokens per LLM call, at high reasoning effort; no operating guide beyond the notes of Section 4.2; and a reminder, at most 20 times per episode, when an LLM call returns no code. One LLM call returns at most one code cell:



23 python(code) 2 primitive calls; an excerpt

```
world=robo.world_map('agentview')                                    composition
...
points=region[(colors.mean(axis=2)>100)&(region[:,:,2]>1.24)&(region
    ↪ [:,:,2]<1.32)][::8]
...
for iteration in range(150):
    sample=points[rng.choice(len(points),3,replace=False)]
    normal=np.cross(sample[1]-sample[0],sample[2]-sample[0])
...
    residual=np.abs((points-sample[0])@normal)
    mask=residual<0.0015
...
```

back: the error it raised, `TypeError`

24 python(code) 2 primitive calls; an excerpt

```
...
center=points[best].mean(axis=0)
_,singular,vectors=np.linalg.svd(points[best]-center,full_matrices=False)
normal=vectors[-1]                                                   composition
...
```

back: the 3 lines it printed, no image

29 python(code) 7 primitive calls; an excerpt

```
...
yaw_delta=math.atan2(actual_normal[0],actual_normal[1])              composition
...
print(robo.rotate_wrist(delta_yaw=yaw_delta,gripper=+1,max_steps=100,step_clip
    ↪ =0.035))
...
pitch_delta=math.atan2(actual_normal[1],actual_normal[2])
...
print(robo.rotate_pitch(delta_pitch=pitch_delta,gripper=+1,max_steps=100,
    ↪ step_clip=0.035))
...
```

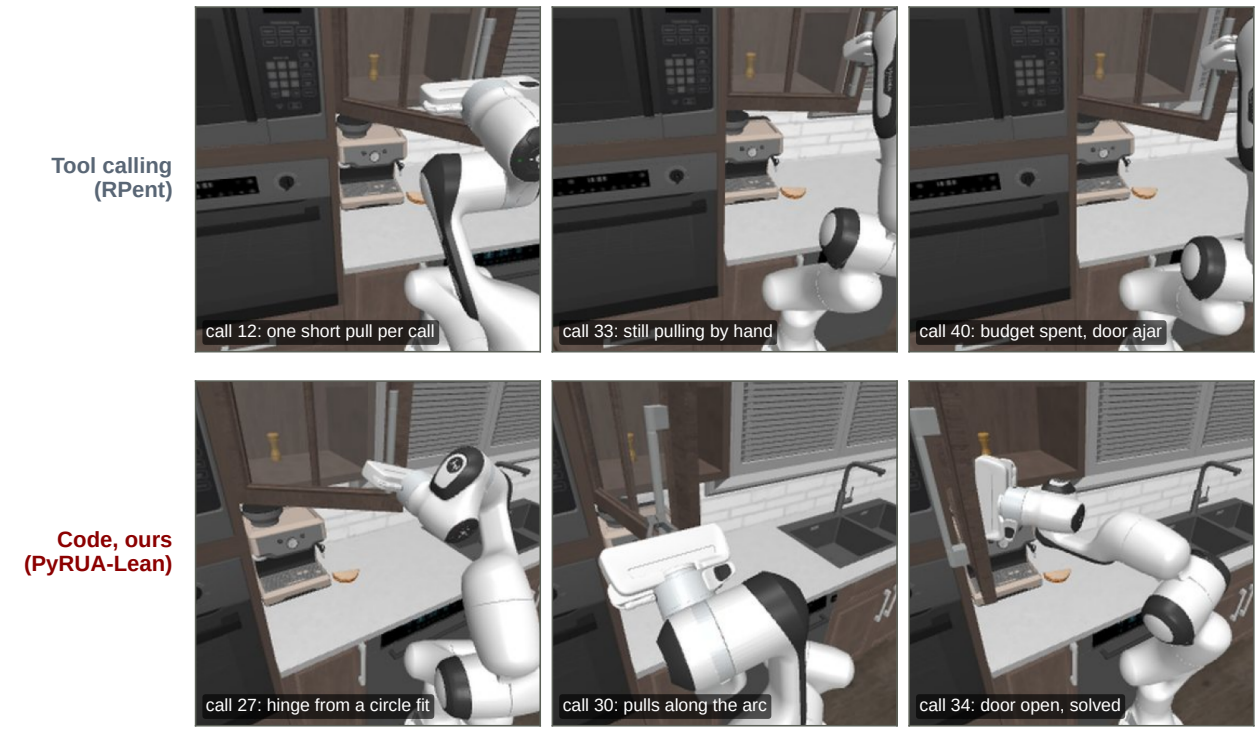
back: the 7 lines it printed and the image it asked for

33 python(code) 10 primitive calls; an excerpt ✓

```
...
actual_normal=rotation_matrix(robo.state().eef_quat)@local_normal
height=initial_knob[2]-anchors['table'][2]
base_center=current_knob-actual_normal*height                        composition
...
place_target=np.array(robo.state().eef_pos)+burner-base_center
place_target[2]+=0.007
...
print(robo.move_to(place_target.tolist(),gripper=+1,step_clip=0.006,max_steps
    ↪ =100,tol=0.006))
...
```

back: the 5 lines it printed, no image

Figure 10 An instance only code solved: LIBERO-PRO, "turn on the stove and put the moka pot on it" (10 swap, task 2, seed 3). Both agents turned the stove on and dropped the pot. Tool calling called the VLA pick primitive (`pi0_pick`, which runs $\pi_{0.5}$; "Pi0" in the labels) on the fallen pot four times and ended the episode; the code agent grasped it itself, fitted a plane to the points of its lid to find how it leans, turned it upright, and set its base, not the gripper, over the burner. *Top:* the recorded camera views after the LLM calls named. *Bottom:* the code agent's cells, excerpts exactly as the model wrote them; "..." marks lines left out. The plane fit in LLM call 23 raised an error (NumPy's linear algebra refuses half-precision arrays); LLM call 24 cast the points and ran it again.



27 python(code) 5 primitive calls; an excerpt

```
...
handle=np.array(sample.median_xyz)
observations=np.array([[0.43,-3.2447],[0.5879,-3.2852],handle[:2]])
center=np.linalg.solve(2*(observations[1:]-observations[0]),np.sum(observations
    ↪ [1:]**2,axis=1)-np.sum(observations[0]**2))
radius=np.linalg.norm(observations[0]-center)                    composition
...
```

back: the 4 lines it printed and the image it asked for

28 python(code) 18 primitive calls; an excerpt

```
current=np.array(robo.state().eef_pos)
arc_radius=np.linalg.norm(current[:2]-center)                    composition
arc_angle=math.atan2(current[0]-center[0],current[1]-center[1])
arc_height=current[2]
...
for increment in range(4):                                       composition
    arc_angle+=0.1
    target=np.array([center[0]+arc_radius*math.sin(arc_angle),center[1]+
    ↪ arc_radius*math.cos(arc_angle),arc_height])
    moved=robo.move_to(target,gripper=robo.CLOSE,step_clip=0.005,max_steps=80,
    ↪ tol=0.005)
...
    if robo.done or not moved.reached or robo.state().gripper_opening<0.008 or
    ↪ robo.state().gripper_opening>0.055:
        break
...
```

back: the 9 lines it printed and the 2 images it asked for

Figure 11 An instance only code solved: RoboCasa365 atomic, "Open the cabinet door." (seed 4). Both agents located the door's hinge. Tool calling pulled the handle a few centimeters per LLM call and was still pulling when its 40 LLM calls ran out; the code agent took the hinge as the center of the circle through three positions of the handle, two of which it copied from earlier outputs, and pulled along that arc in one cell, stopping if a move fell short or the grip slipped. The door opened, and a VLA call in LLM call 34 completed the task. *Top:* the recorded camera views; *bottom:* the code agent's cells, excerpts exactly as the model wrote them.

in the RGB configuration an episode runs 22.6 cells in its 25.7 LLM calls on average, and the remaining calls end the episode or return text only.

Checks on the evaluation runs. For all four seeds (one of the RGB configuration, three with depth), we checked that all 54 task names (a Generalization task and its randomized twin are two names) ran the protocol's number of episodes on distinct layouts, that the three camera videos of every episode exist, that no episode ended on an error or a timeout of the evaluation client, and that the verdict our adapter recorded agrees with the client's own result file for every episode. The board-style values we compute from the episodes equal those of RoboDojo's own summary script.

I RoboDojo with depth: what differs

The configuration of Section 4.4 is the agent of Section 4 with one change to its observation and to the three primitives that read it (Table 15). The planner, the agent loop, the budgets, the motion and gripper primitives, and the rest of the prompt are the same. Its success rate on each task is the third column of Table 17. In this configuration an episode takes 23.9 LLM calls on average, and the three seeds cost about $14,000 at our model gateway, or $2.2 per episode.

Table 15 What differs between the two configurations of the RoboDojo agent.

	RGB (Section 4)	RGB and depth (Section 4.4)
Observation	Three RGB images and the robot state.	The same, plus a depth image and the intrinsic and extrinsic matrices of each camera, which are options of the client's observation configuration.
`locate`	Takes the mask of the object in the camera asked, looks for the same object in the other two cameras, and reports the footprint and height that fit every camera that shows it. With one camera the height is a guess, and the result says so.	Takes the 3D points of the depth pixels under the mask, from one camera: they give center, top, and footprint directly.
`back_project`	Segments the object under the pixel and measures it as `locate` does; the answer is where the pixel's ray meets it, or the table.	Reads the depth at the pixel.
`camera_meta`	Worked out from the fixed calibration of the station and, for a wrist camera, from the arm's state.	The matrices the client sends.
Operating notes	Say how to measure with the wrist cameras: what one camera cannot measure, and to look again from above before a grasp.	Not needed.

J PyRUA-Lean Coder: training settings

Table 16 lists the settings with which PyRUA-Lean Coder v0.1-4B was trained (Section 5.1).

Table 16 Training settings of PyRUA-Lean Coder v0.1-4B.

Base model	Qwen3.5-4B, fine-tuned in full
Algorithm	GRPO; clip range 0.2 below, 0.28 above; no KL term, no entropy term
Reward	1 if the benchmark's success check passes, 0 otherwise
Training instances	45: LIBERO-PRO goal, spatial and object suites, 3 tasks × 5 seeds each
Episodes per iteration	16 instances × 8 rollouts = 128
Optimizer steps per iteration	2, of 64 episodes each
Optimizer	Adam, learning rate 10^{-6} (constant), $\beta = (0.9, 0.98)$, weight decay 0.1
Iterations	30 (3,840 episodes, 60 optimizer steps)
Rollout sampling	temperature 1.0
Limits of a training episode	24 code cells, 30 LLM calls, 25 minutes; 4,096 new tokens per LLM call
Generated tokens per episode	at most 40,960; context 65,536 tokens
Hardware and software	one node with eight GPUs; miles for training, SGLang for rollouts; about twelve hours

K RoboDojo: results per task

Table 17 lists the 42 tasks of RoboDojo with the success rate of PyRUA-Lean, of PyRUA-Lean with depth (Section 4.4), of GPT-6 Astra controlling the end-effector directly, and of the best entry of the public board on that task. PyRUA-Lean is above every entry on 22 tasks and level with the best on 3.

Table 17 Success rate (%) per RoboDojo task: PyRUA-Lean (one evaluation seed), PyRUA-Lean with depth (three evaluation seeds; Section 4.4, not compared with the board), GPT-6 Astra with direct end-effector control [47] (*Astra*), and the best entry of the public board dated October 9, 2026 on that task (*Best*, among the 57 policies with per-task results; where several hold it, their number is given, and a dash where it is 0.0). A Generalization task's value pools its standard and randomized episodes. Bold: PyRUA-Lean above every entry of the board.

Task	Ours	Ours, with depth	Astra	Best	Best board entry
Generalization (12 tasks)					
stack_bowls	**78.0**	75.3	62.0	74.7	Simate-beta
push_T	**76.0**	78.0	60.0	60.0	GPT-6 Astra (direct)
pack_objects_into_box	**32.0**	17.3	0.0	17.3	Simate-beta
fold_clothes	70.0	48.7	72.0	78.0	Simate-beta
hang_mugs	2.0	14.0	0.0	32.0	VPP2
sweep_blocks	**18.0**	24.7	4.0	14.0	Rex-M1 Preview
pour_liquid_into_cup	8.0	16.0	4.0	67.3	Rex-M1 Preview
make_toast	6.0	18.7	0.0	39.3	VPP2
arrange_largest_number	**86.0**	76.7	60.0	60.0	GPT-6 Astra (direct)
sort_nesting_dolls_by_size	**74.0**	76.0	18.0	18.7	VPP2
store_laptop_and_headphones	0.0	0.0	0.0	34.0	Simate-beta
stack_blocks	**100.0**	100.0	86.0	86.0	GPT-6 Astra (direct)
Precision (8 tasks)					
fasten_screws	**66.0**	69.3	16.0	48.0	Awomo-0.5
plug_in_charger	12.0	14.0	0.0	30.7	PhysicalRSI
insert_tubes	44.0	70.7	0.0	74.7	VPP2
pour_balls_into_vase	**76.0**	69.3	4.0	60.7	VPP2
play_Xylophone	**64.0**	67.3	8.0	30.7	Awomo-0.5
deposit_coin	34.0	34.0	2.0	56.0	PhysicalRSI
insert_key	0.0	0.0	0.0	39.3	PhysicalRSI
build_tower	2.0	4.0	2.0	83.3	Simate-beta
Long-Horizon (8 tasks)					
put_bottles_into_dustbin	76.0	94.7	10.0	100.0	Simate-beta
fill_pen_holder	14.0	16.7	0.0	42.0	InternW0-Δ
classify_objects	**98.0**	99.3	56.0	56.0	GPT-6 Astra (direct)
play_tic_tac_toe	94.0	94.7	0.0	94.0	PhysicalRSI
fill_egg_holder	**2.0**	2.0	0.0	1.0	Liber-0 Lite
organize_table	**34.0**	39.3	0.0	26.0	Simate-beta
make_kong	0.0	0.0	0.0	90.0	GalaxeaVLA (G0.5)
play_stacking_toy	**66.0**	67.3	0.0	23.3	VPP2
Memory (6 tasks)					
cover_blocks	100.0	100.0	46.0	100.0	8 entries
match_and_pick_from_conveyor	82.0	86.7	62.0	92.0	InternW0-Δ
swap_blocks	0.0	0.0	0.0	38.7	Simate-beta
swap_T	**98.0**	100.0	18.0	92.7	PhysicalRSI
press_by_number	0.0	0.0	70.0	96.0	VPP2
imitate_sorting_sequence	**98.0**	97.3	36.0	36.0	GPT-6 Astra (direct)
Open (8 tasks)					
align_blocks	78.0	71.3	50.0	86.0	PhysicalRSI
general_pickup	**98.0**	93.3	84.0	84.0	GPT-6 Astra (direct)
stack_blocks_by_language	**100.0**	100.0	40.0	40.0	GPT-6 Astra (direct)
solve_equation	**90.0**	86.7	40.0	60.0	PhysicalRSI
classify_objects_by_language	**92.0**	90.7	30.0	30.0	GPT-6 Astra (direct)
pick_from_conveyor_by_image	**34.0**	28.7	4.0	4.0	GPT-6 Astra (direct)
store_tools_in_toolbox	0.0	0.0	0.0	0.0	–
pour_by_language	**20.0**	27.3	0.0	0.0	–